

ALGORITMOS DE ORDENAMIENTO AVANZADOS

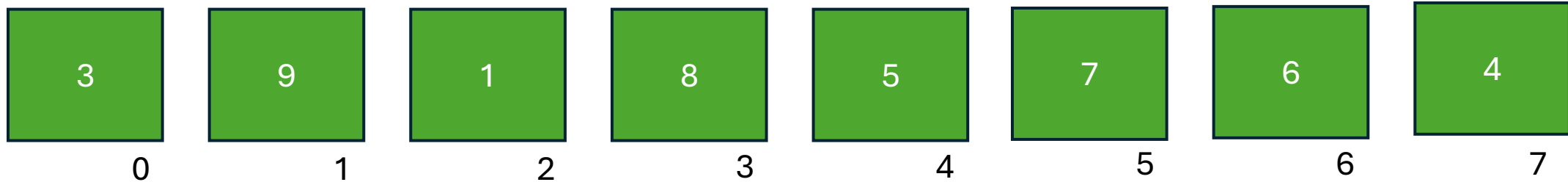
Shell - Quick

Profesor: Ing. Weber Federico

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 } ;
```

N = 8



Es una mejora del método por inserción.

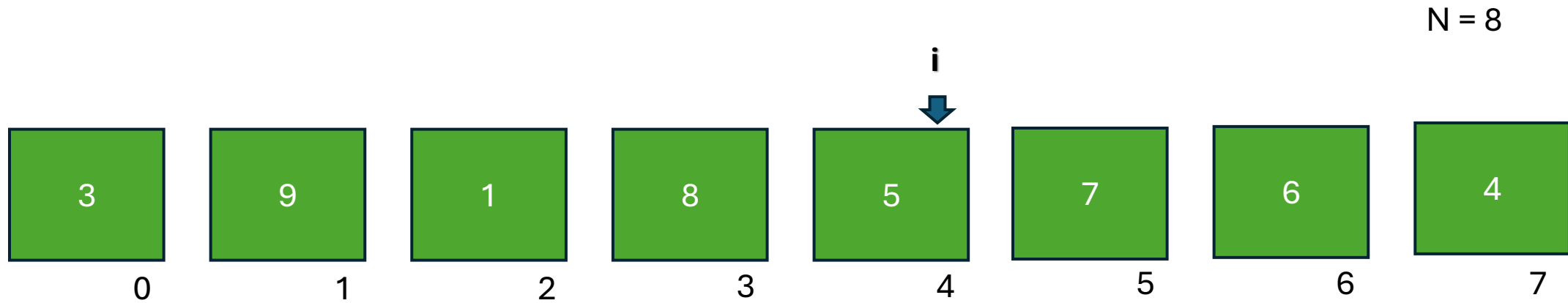
En el algoritmo de inserción, cada elemento se compara con los elementos de la izquierda uno por uno. Si el elemento que queremos insertar es el menor, puede tomar bastante tiempo antes de encontrar su lugar.

Pero el algoritmo de Shell hace las cosas de manera un poco diferente: en lugar de comparar uno por uno, da **pasos** más grandes al principio y luego los reduce gradualmente. Esto hace que la ordenación sea más rápida.

Por lo general, comienza dando **saltos de tamaño $n/2$ (n es el número de elementos)**, y luego en **cada iteración reduce a la mitad el tamaño de los saltos**, hasta que llega a saltos de tamaño 1.

Shell sort

```
int vec[8] = { 3 , 9 , 1 , 8 , 5, 7, 6, 4 };
```

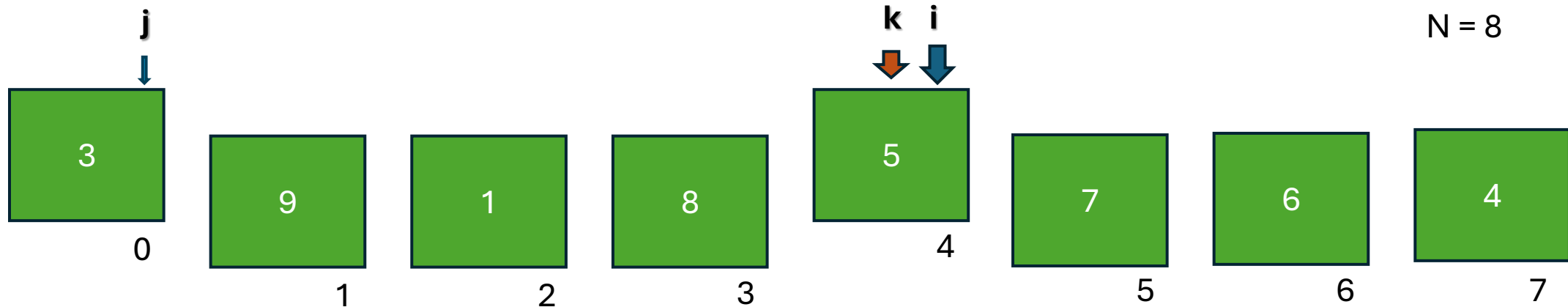


Paso inicial = $N / 2$

Paso inicial = 4

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```



If (3 > 5)

swap

Paso inicial = $N / 2$

Paso inicial = 4

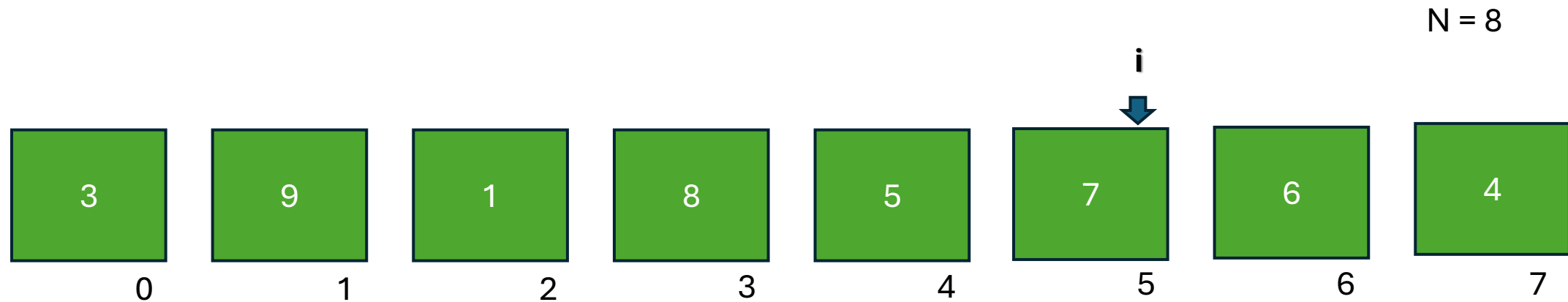
i comienza en paso

j comienza en i -paso

k comienza en j +paso

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```

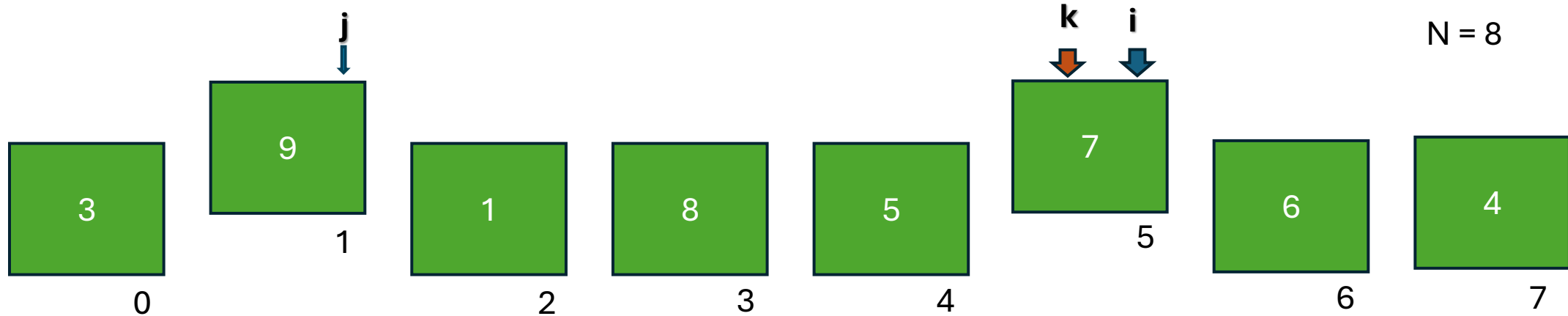


Paso inicial = $N / 2$

Paso inicial = 4

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```



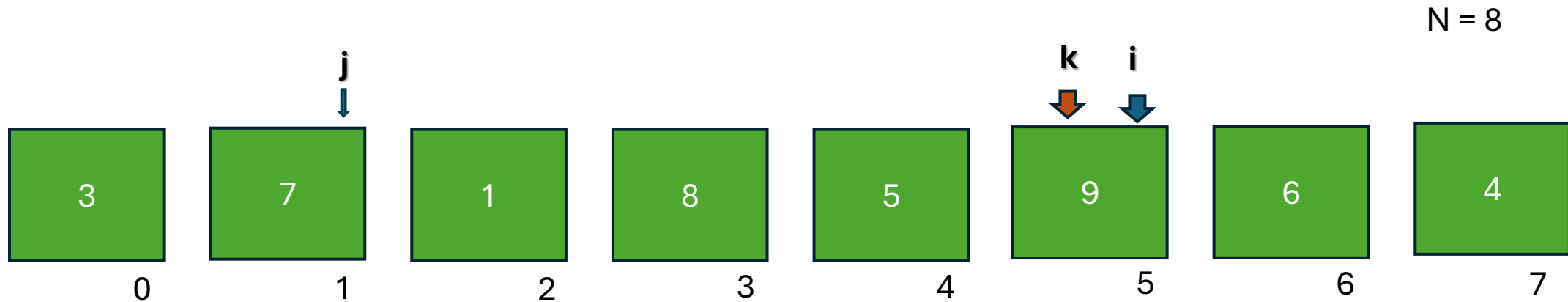
If (9 > 7)
swap

Paso inicial = $N / 2$

Paso inicial = 4

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```



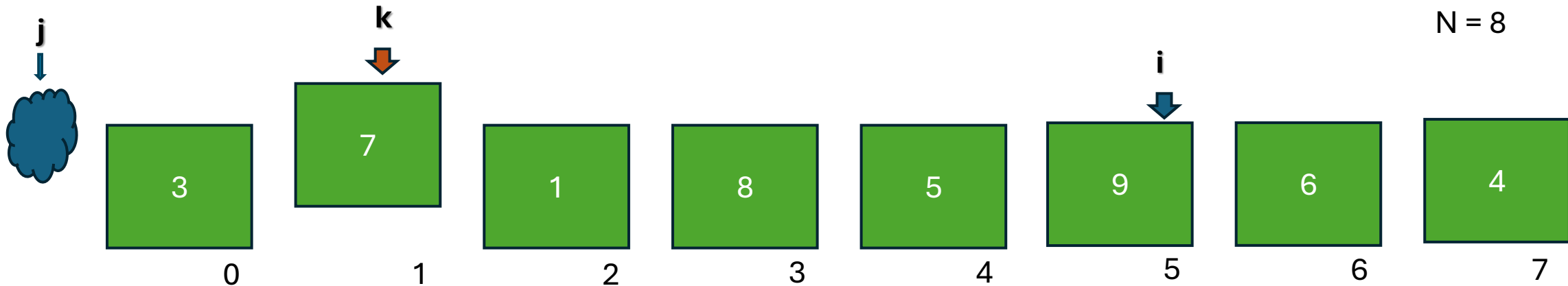
If (9 > 7)
swap

Paso inicial = $N / 2$

Paso inicial = 4

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 } ;
```



Luego del intercambio se debe verificar que el 7 haya quedado insertado correctamente. Para esto se lo debe comparar con el que este 4 posiciones a la izquierda, que en este caso no existe.

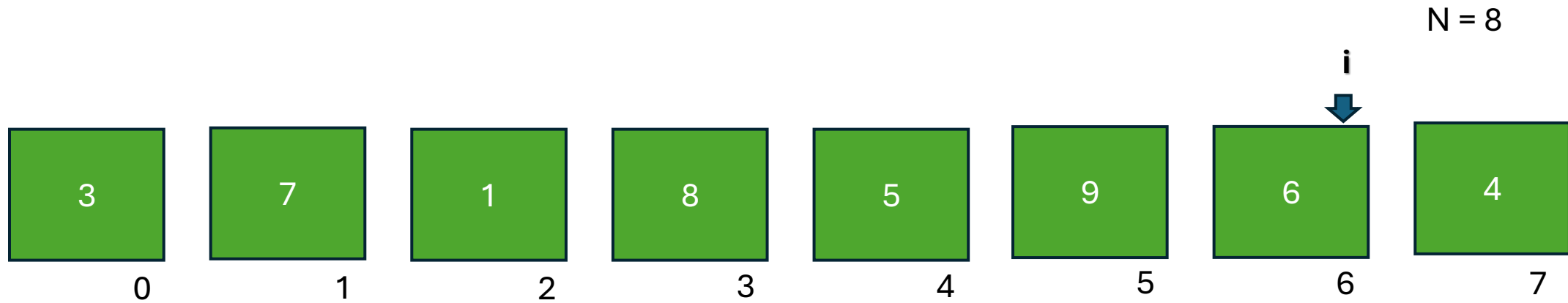
Paso inicial = $N / 2$

Paso inicial = 4

$j < 0$ indica que no hay mas comparaciones

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```



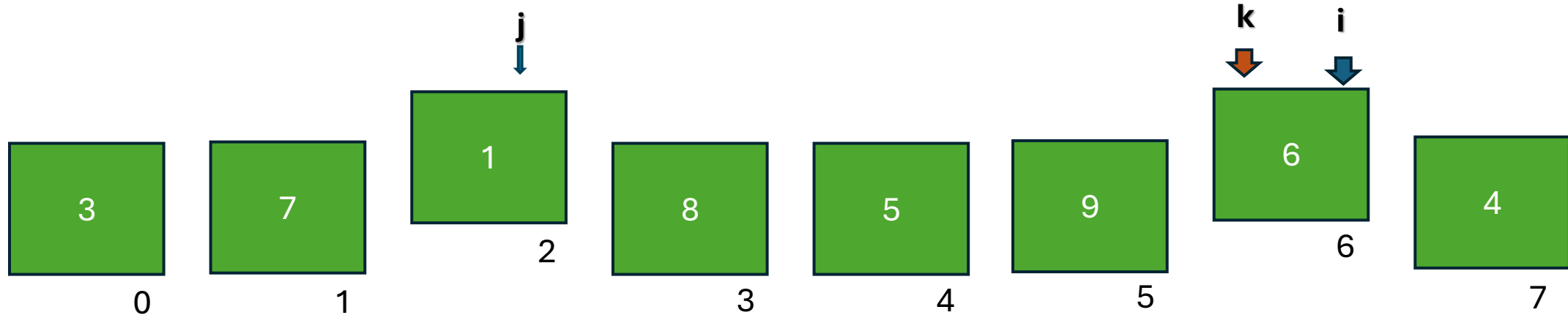
Paso inicial = $N / 2$

Paso inicial = 4

Shell sort

N = 8

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```



If (1 > 6)
swap

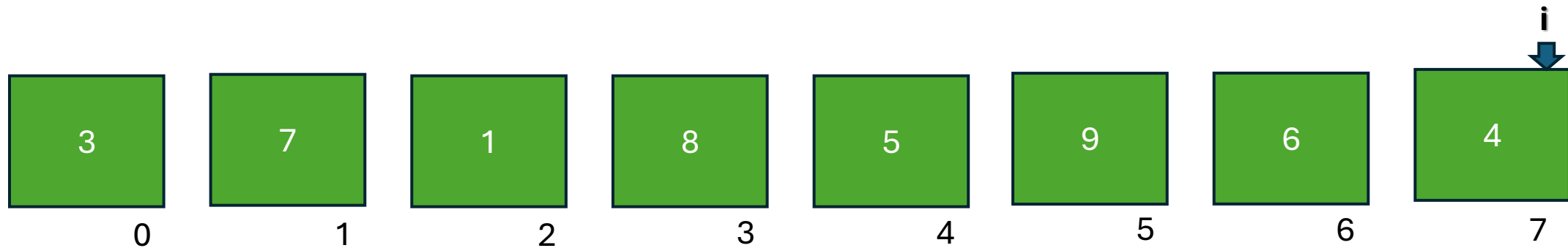
Paso inicial = $N / 2$

Paso inicial = 4

Shell sort

N = 8

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```



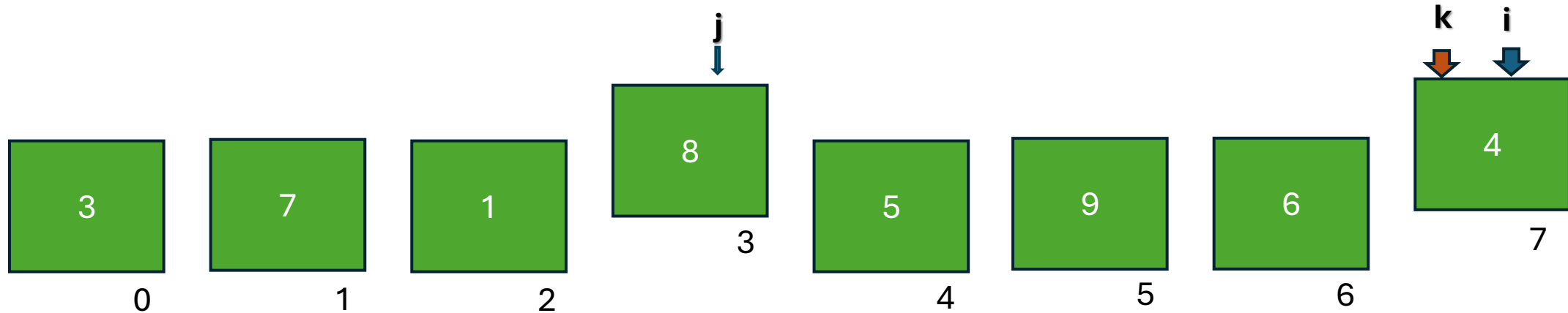
Paso inicial = $N / 2$

Paso inicial = 4

Shell sort

N = 8

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```



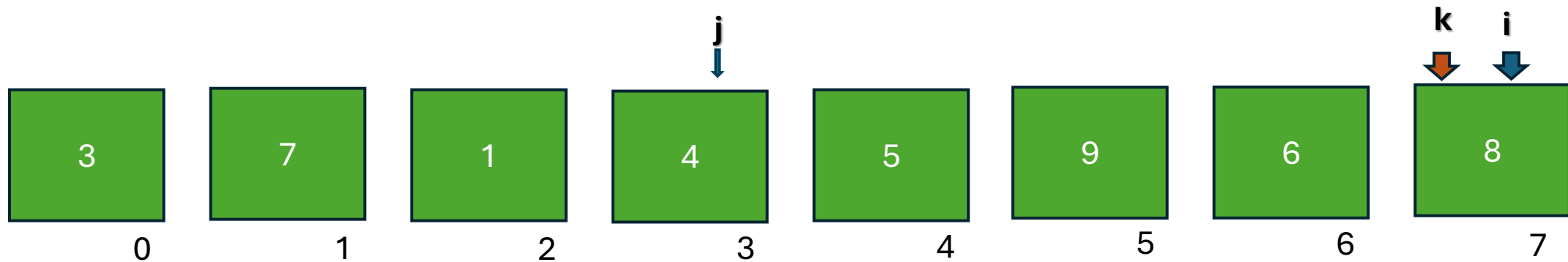
If (8 > 4)
swap

Paso inicial = $N / 2$

Paso inicial = 4

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```



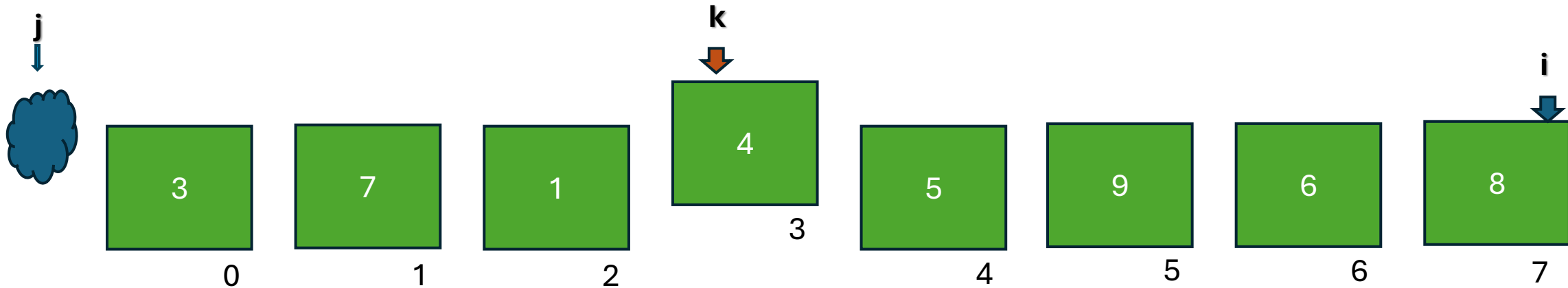
If (8 > 4)
swap

Paso inicial = $N / 2$

Paso inicial = 4

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 } ;
```



Luego del intercambio se debe verificar que el 4 haya quedado insertado correctamente. Para esto se lo debe comparar con el que este 4 posiciones a la izquierda, que en este caso no existe.

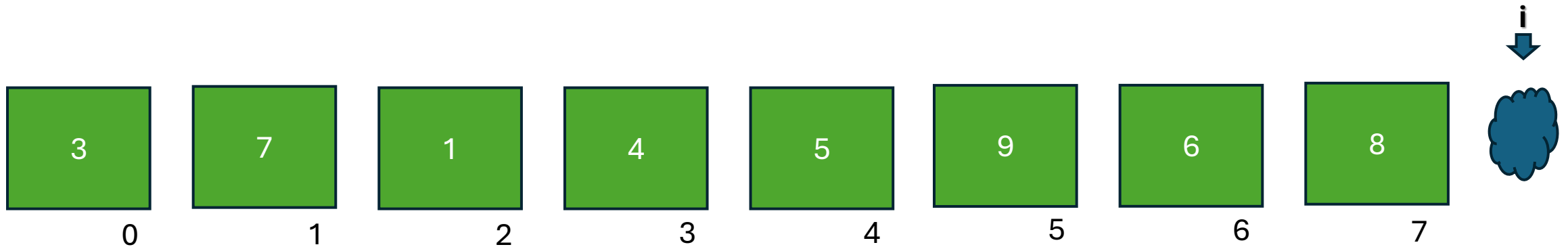
Paso inicial = $N / 2$

Paso inicial = 4

$j < 0$ indica que no hay mas comparaciones

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 } ;
```



Al terminar de recorrer la estructura se debe recalcular el paso y volver a comenzar

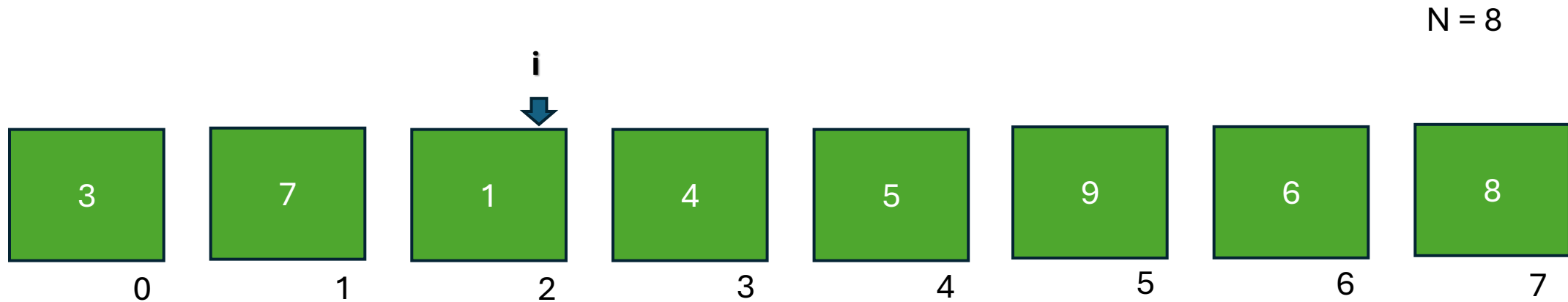
Paso inicial = $N / 2$

Paso inicial = 4

! finaliza al recorrer el ultimo elemento del vector

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```



Al terminar de recorrer la estructura se debe recalcular el paso y volver a comenzar

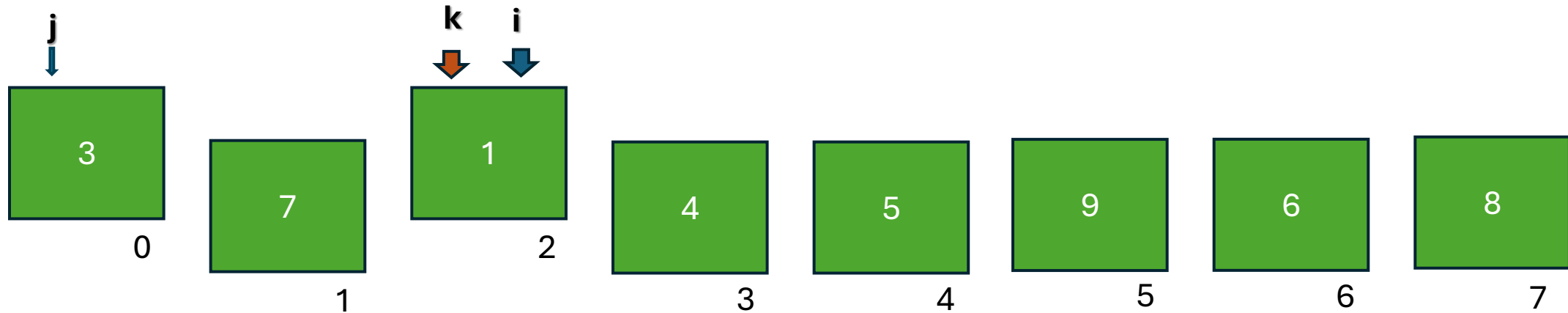
Paso = (paso anterior) / 2

Paso = 2

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```

N = 8



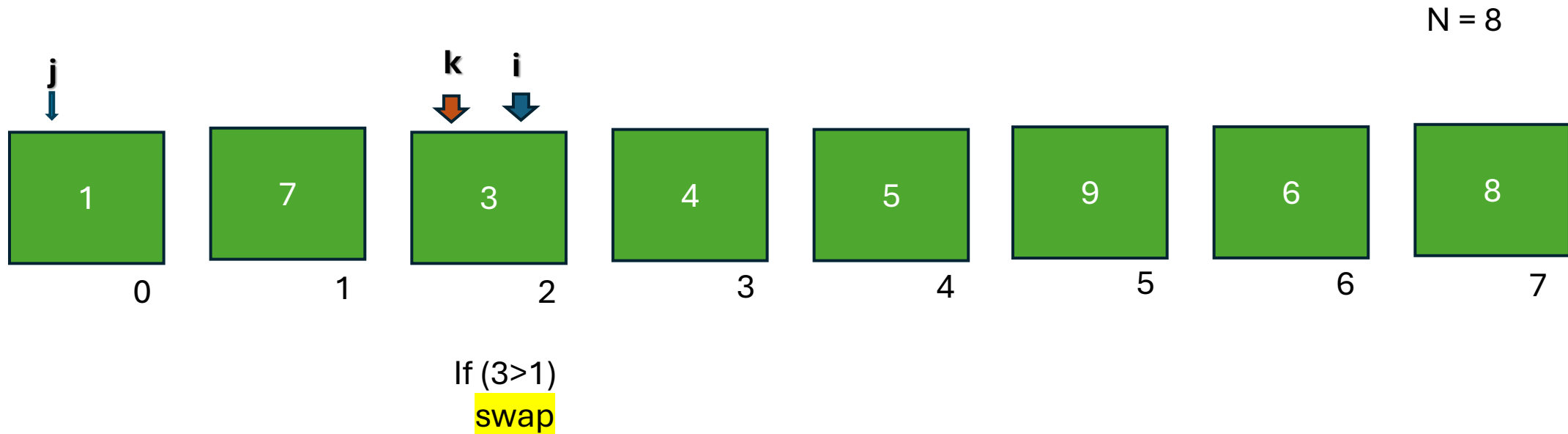
If (3 > 1)
swap

Paso = (paso anterior) / 2

Paso = 2

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```

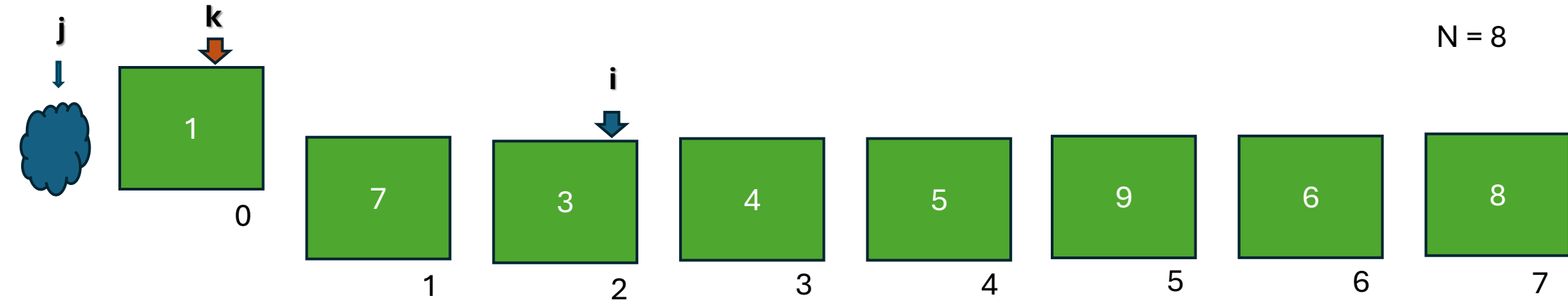


Paso = (paso anterior) / 2

Paso = 2

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 } ;
```



Luego del intercambio se debe verificar que el 1 haya quedado insertado correctamente. Para esto se lo debe comparar con el que este 2 posiciones a la izquierda, que en este caso no existe.

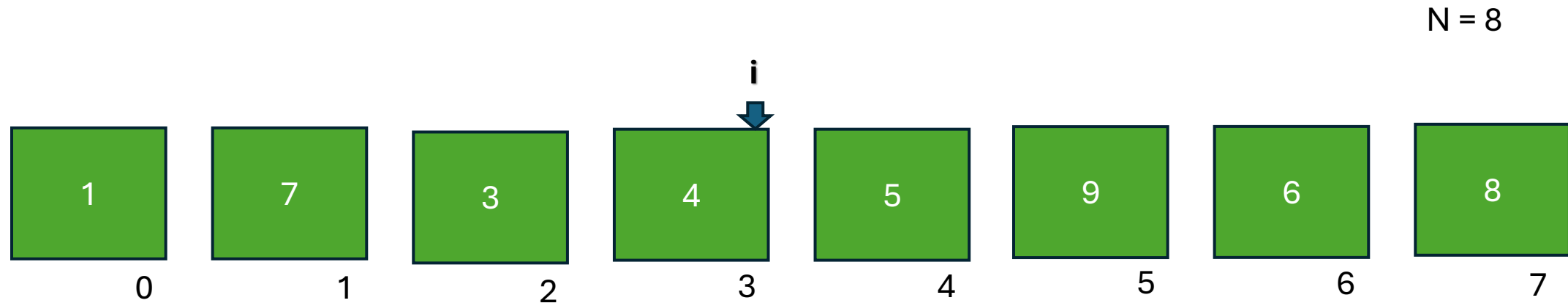
Paso = (paso anterior) / 2

Paso = 2

j < 0 indica que no hay mas comparaciones

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```

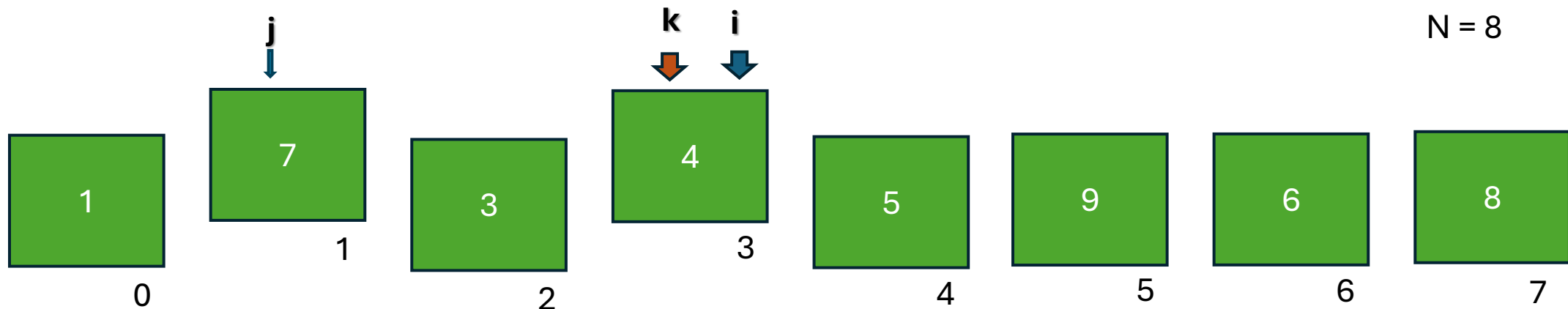


Paso = (paso anterior) / 2

Paso = 2

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 } ;
```

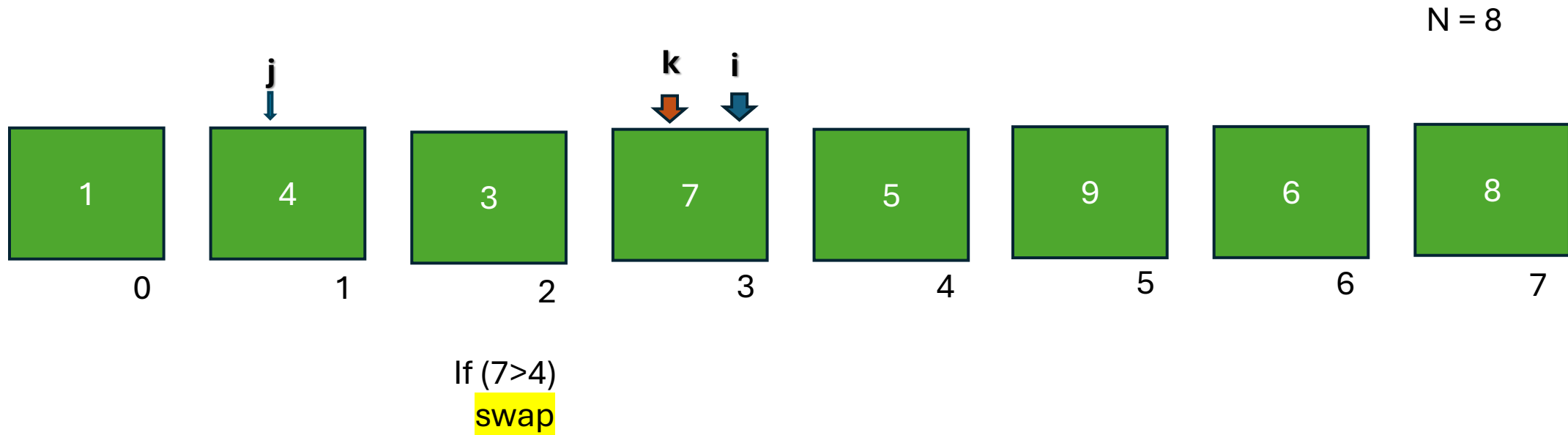


If (7 > 4)
swap

Paso = (paso anterior) / 2

Paso = 2

Shell sort

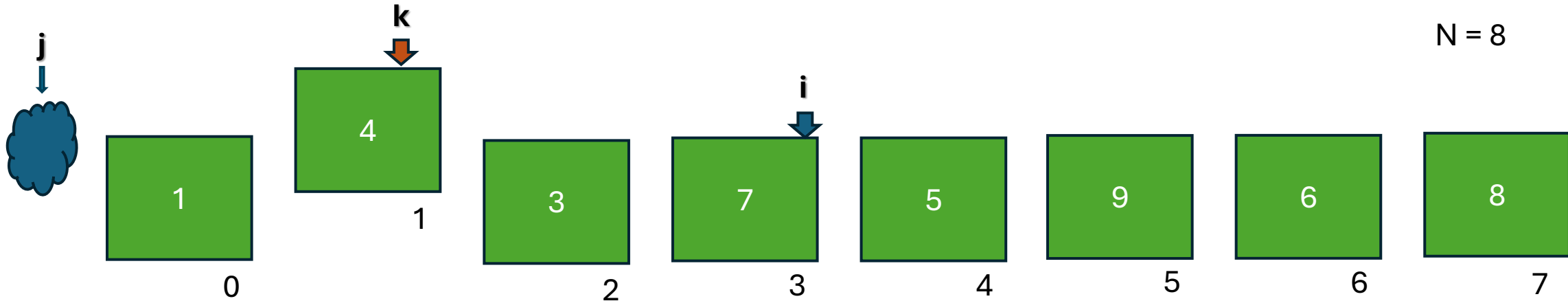


Paso = (paso anterior) / 2

Paso = 2

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 } ;
```



Luego del intercambio se debe verificar que el 4 haya quedado insertado correctamente. Para esto se lo debe comparar con el que este 2 posiciones a la izquierda, que en este caso no existe.

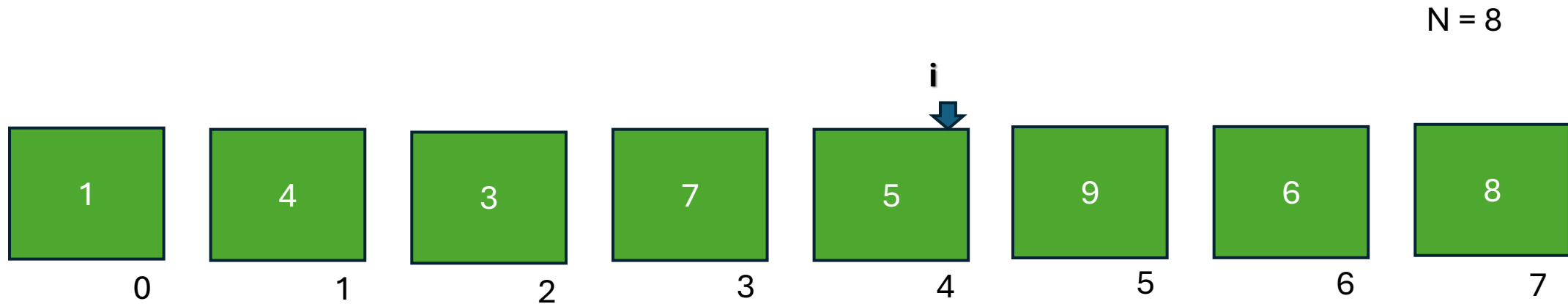
Paso = (paso anterior) / 2

Paso = 2

j < 0 indica que no hay mas comparaciones

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```

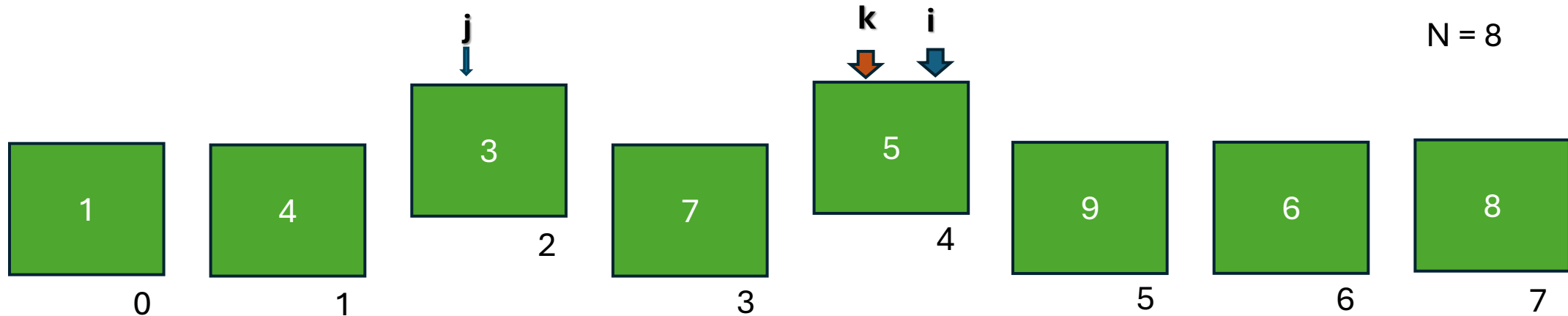


Paso = (paso anterior) / 2

Paso = 2

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 } ;
```



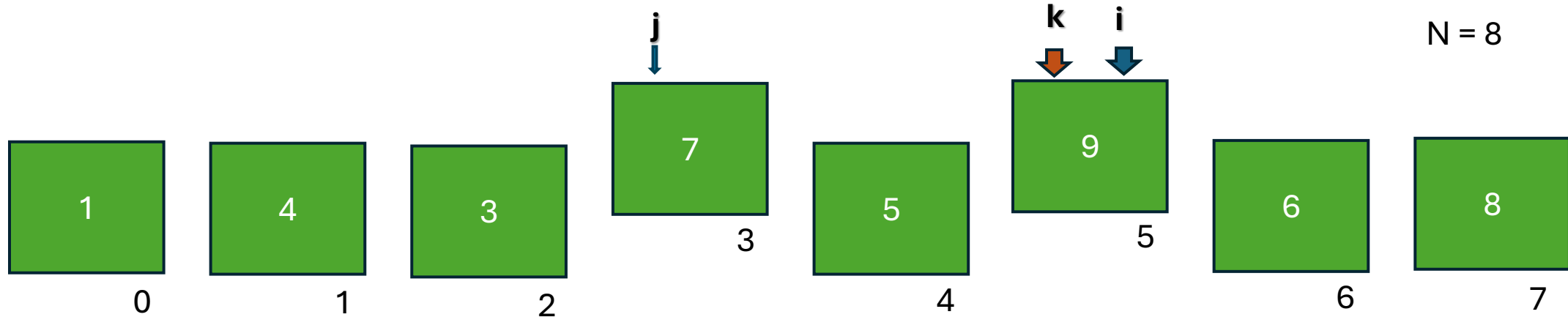
If (3 > 5)
swap

Paso = (paso anterior) / 2

Paso = 2

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```

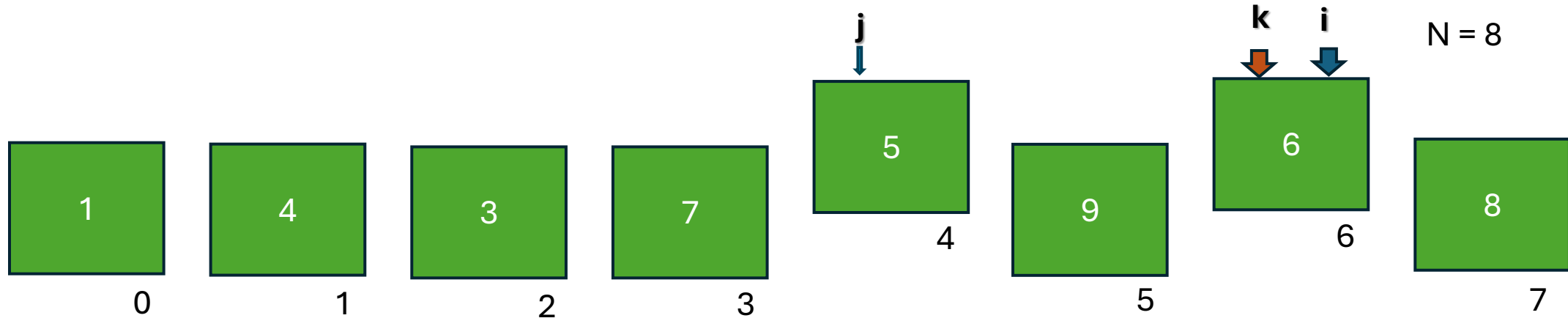


If (7 > 9)
swap

Paso = (paso anterior) / 2
Paso = 2

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 } ;
```



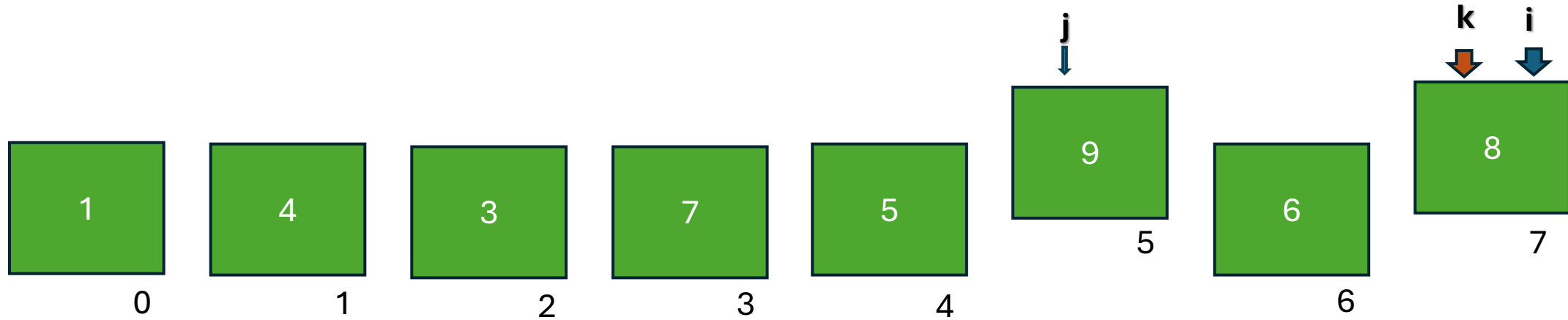
If (5 > 6)
swap

Paso = (paso anterior) / 2

Paso = 2

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```



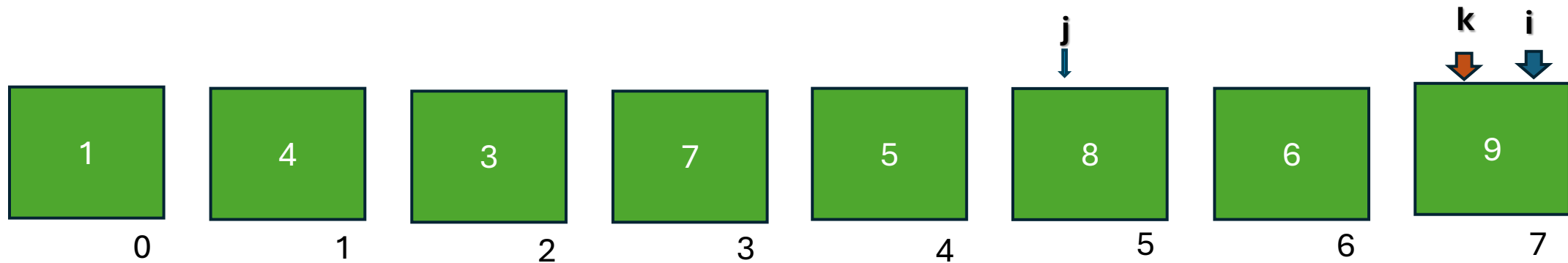
If (9 > 8)
swap

Paso = (paso anterior) / 2

Paso = 2

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```



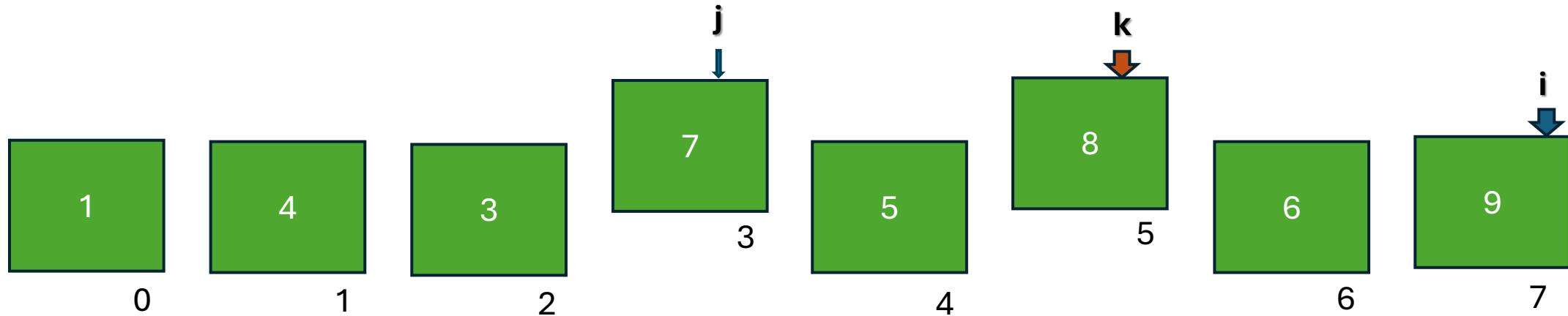
If (9 > 8)
swap

Paso = (paso anterior) / 2

Paso = 2

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```



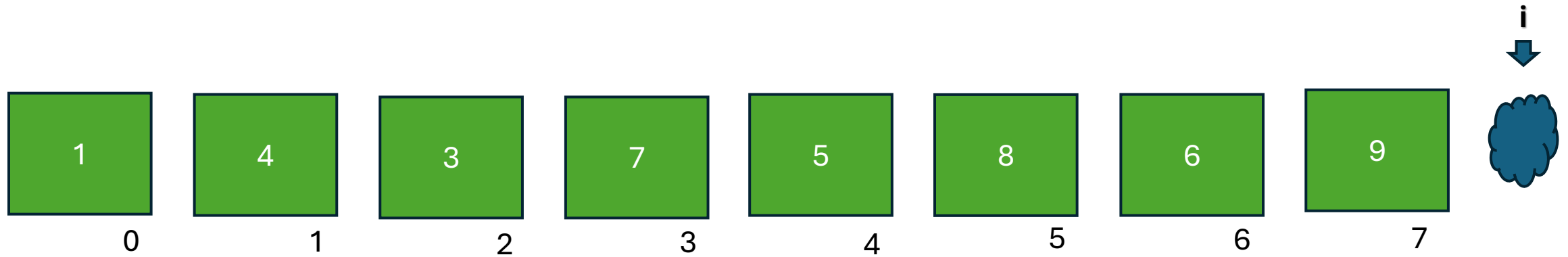
If (7>8)
swap

Paso = (paso anterior) / 2

Paso = 2

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 } ;
```

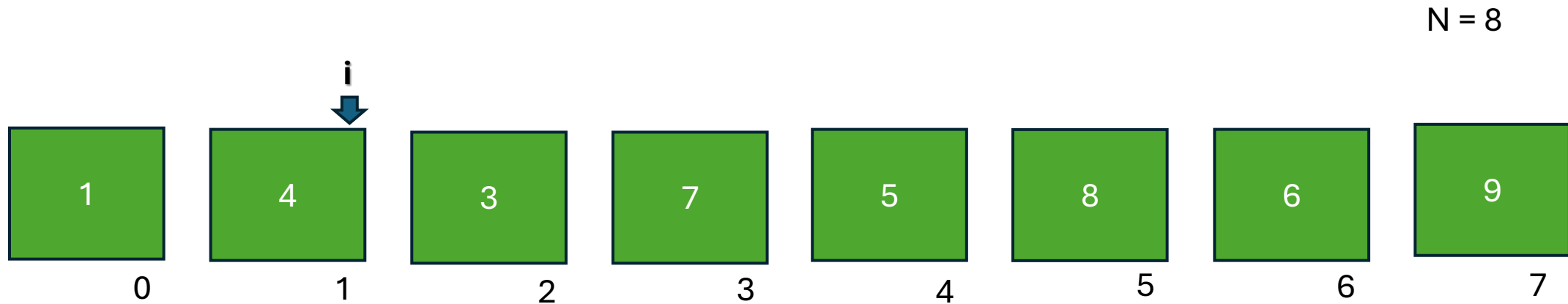


Al terminar de recorrer la estructura se debe recalcular el paso y volver a comenzar

! finaliza al recorrer el ultimo elemento del vector

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 } ;
```



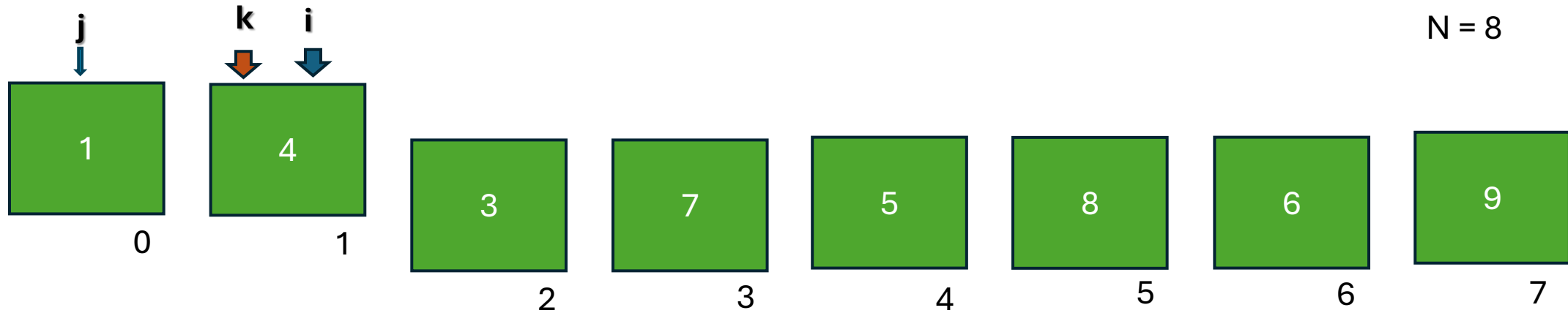
Al terminar de recorrer la estructura se debe recalcular el paso y volver a comenzar

Paso = (paso anterior) / 2

Paso = 1

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 } ;
```



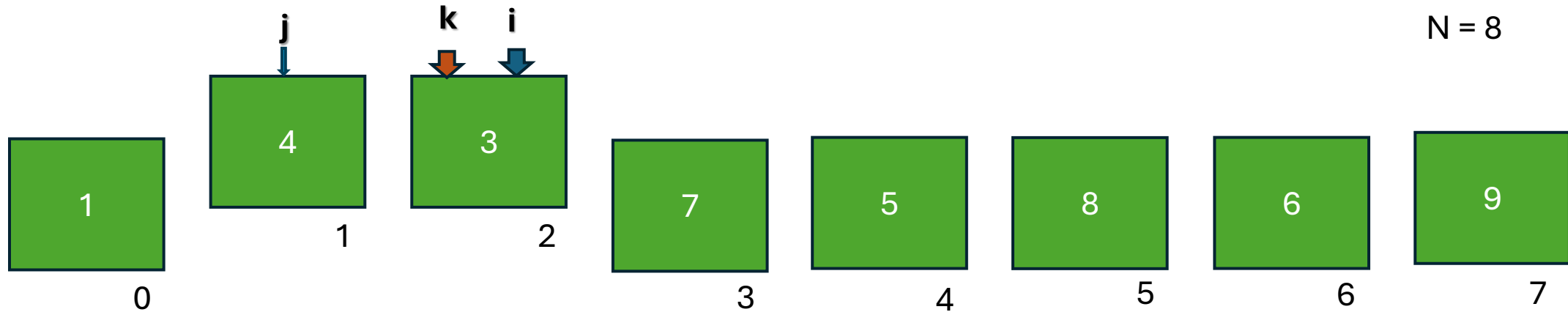
If $(1 > 4)$
swap

Paso = (paso anterior) / 2

Paso = 1

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 } ;
```



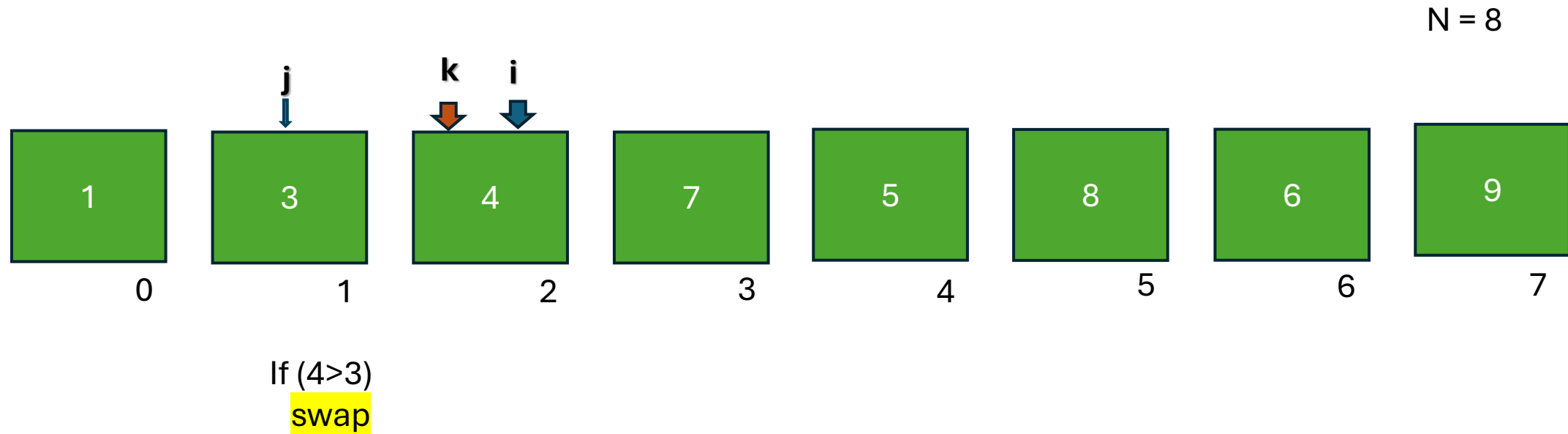
If (4 > 3)
swap

Paso = (paso anterior) / 2

Paso = 1

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 } ;
```

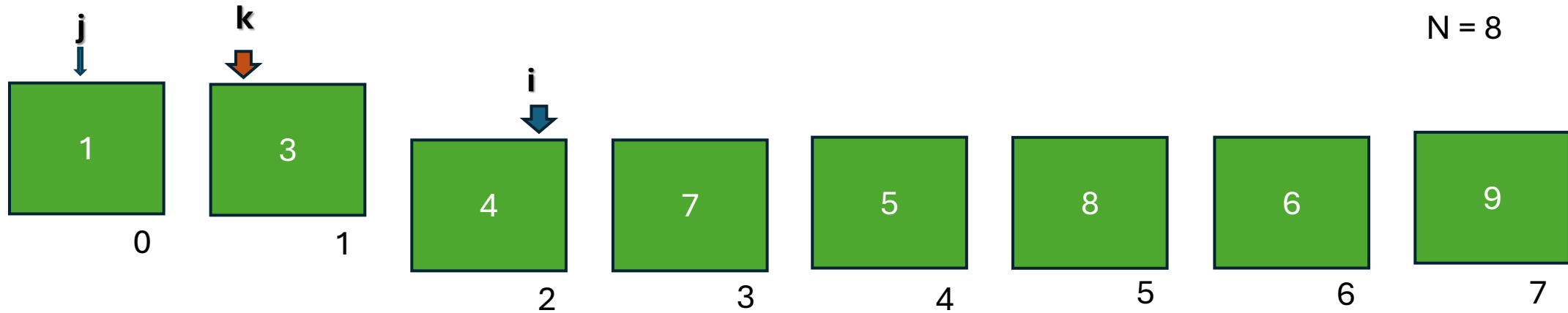


Paso = (paso anterior) / 2

Paso = 1

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 } ;
```



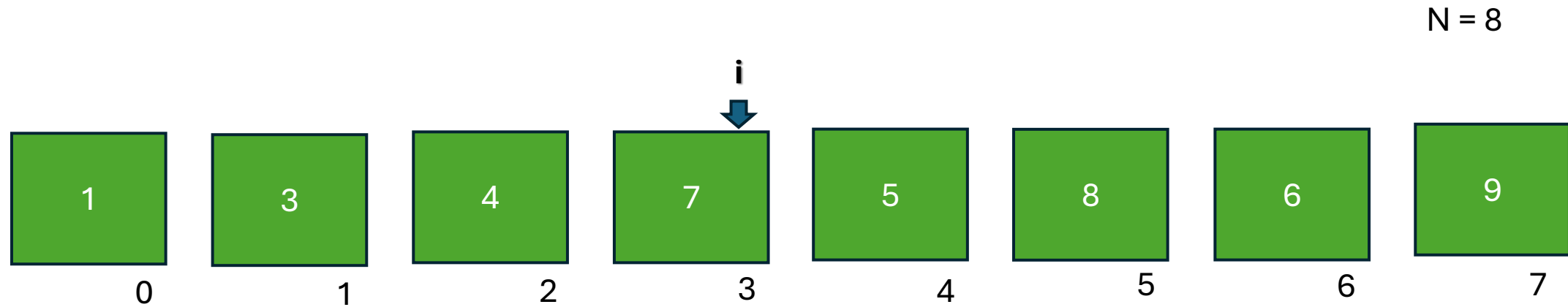
If (1>3)
swap

Paso = (paso anterior) / 2

Paso = 1

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```

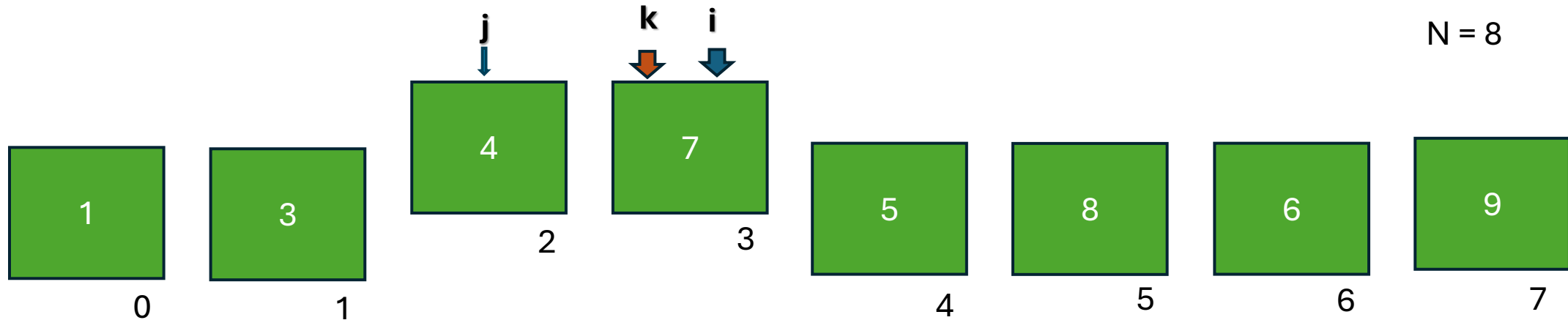


Paso = (paso anterior) / 2

Paso = 1

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 } ;
```



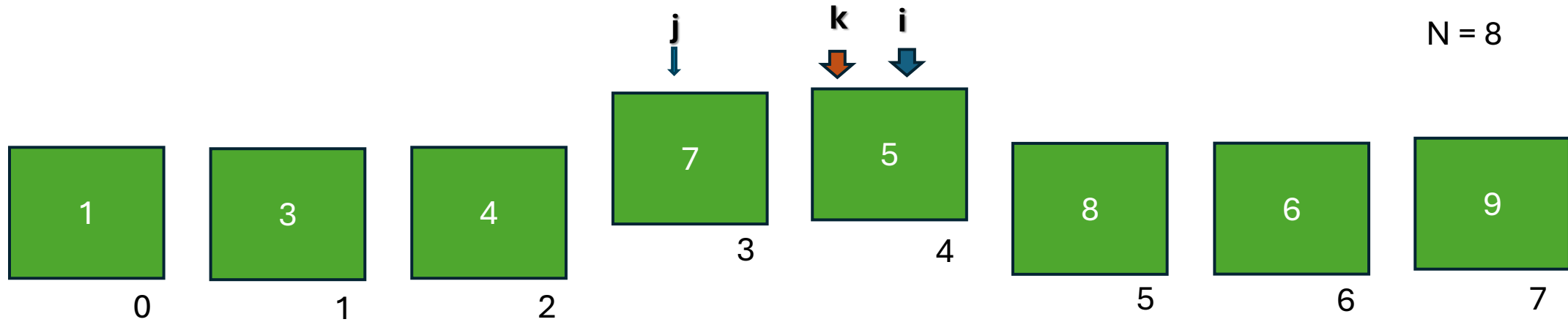
If (4 > 7)
swap

Paso = (paso anterior) / 2

Paso = 1

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```



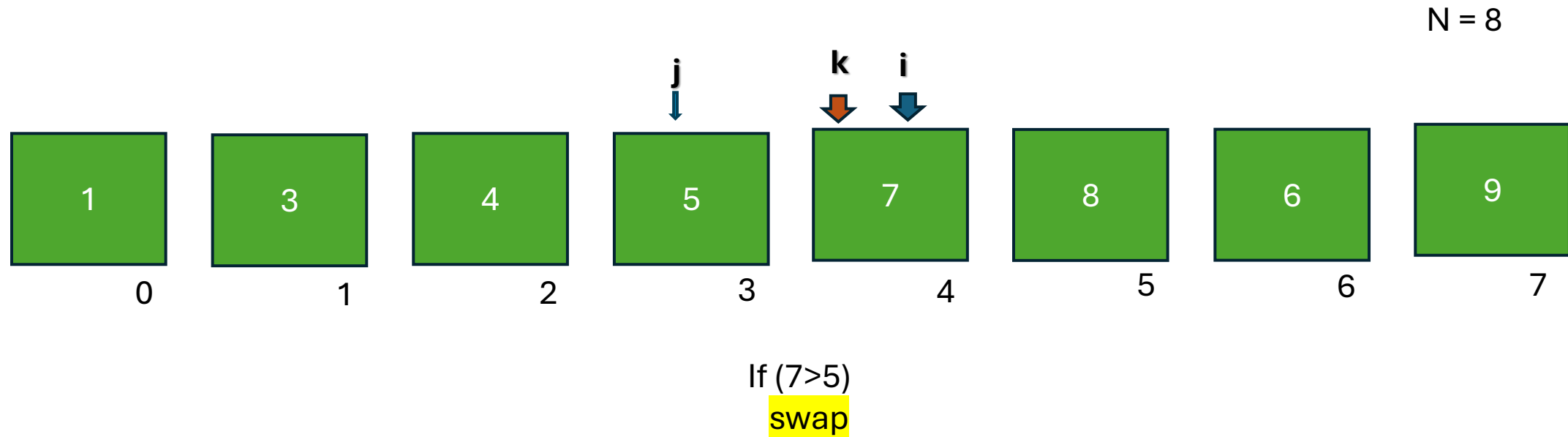
If (7 > 5)
swap

Paso = (paso anterior) / 2

Paso = 1

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 } ;
```

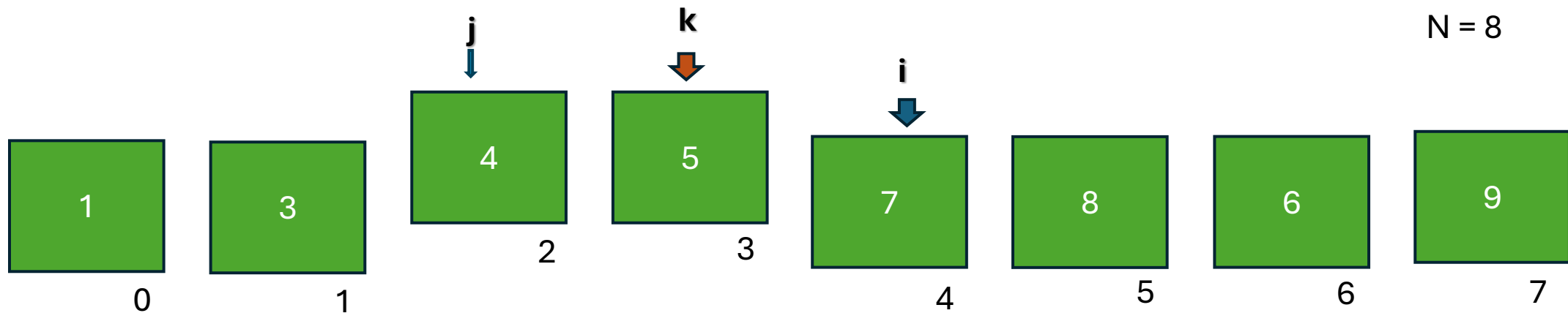


Paso = (paso anterior) / 2

Paso = 1

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 } ;
```



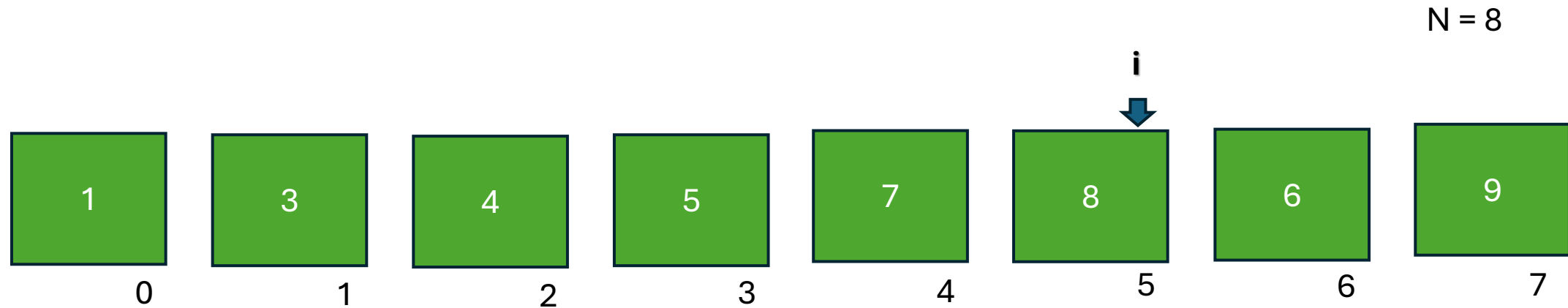
If (4 > 5)
swap

Paso = (paso anterior) / 2

Paso = 1

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```

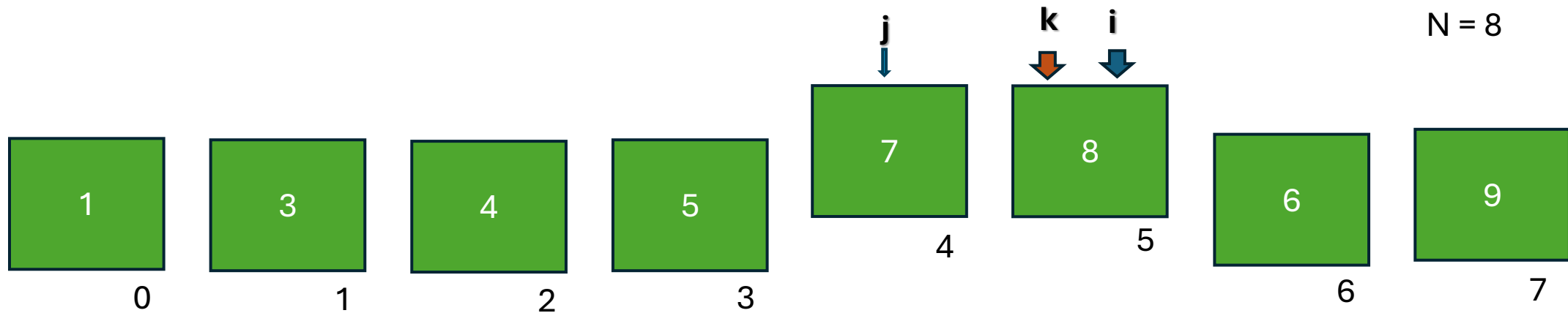


Paso = (paso anterior) / 2

Paso = 1

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```



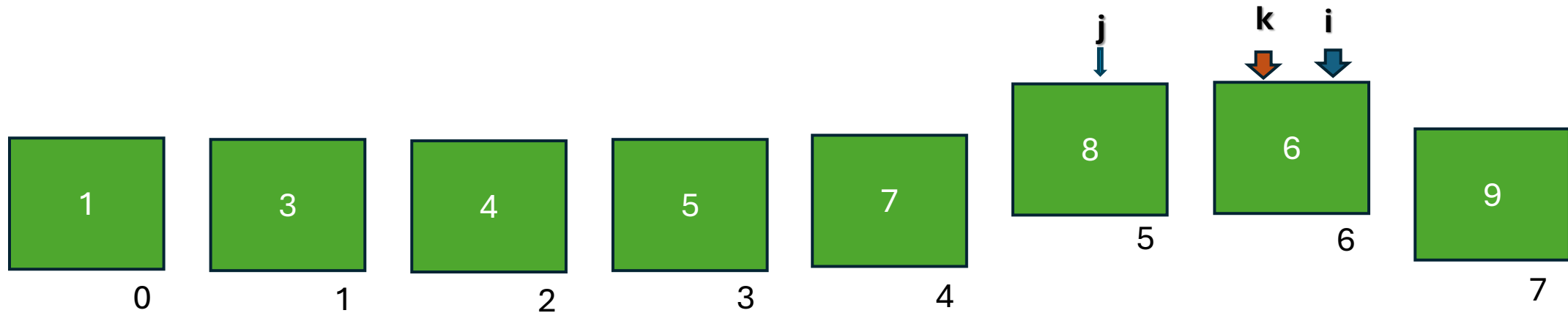
If (7 > 8)
swap

Paso = (paso anterior) / 2

Paso = 1

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```



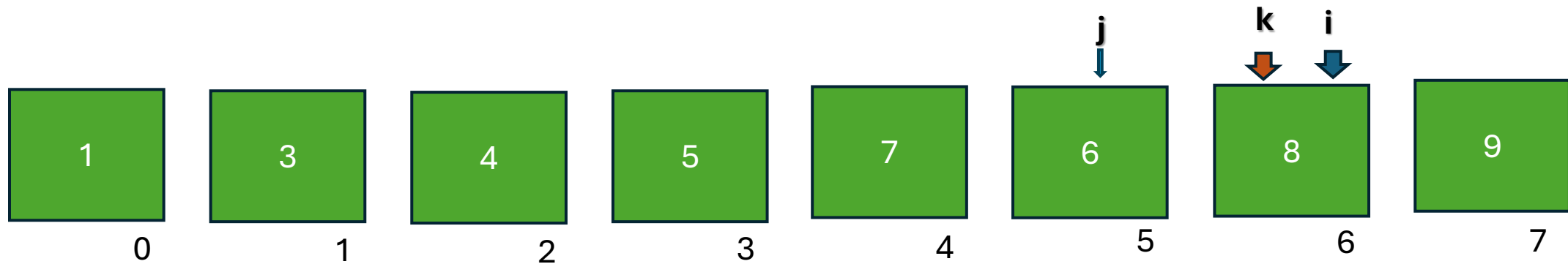
If (8>6)
swap

Paso = (paso anterior) / 2

Paso = 1

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```



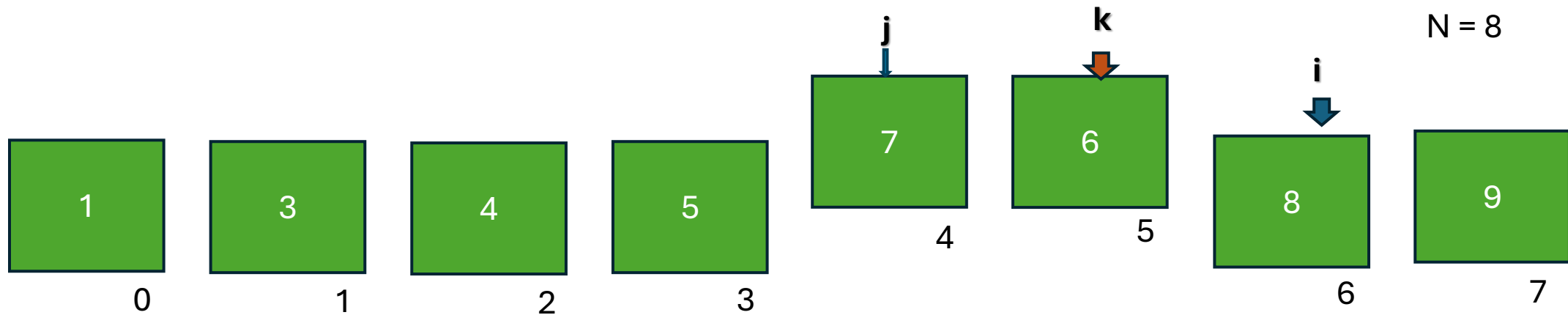
If (8 > 6)
swap

Paso = (paso anterior) / 2

Paso = 1

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 } ;
```



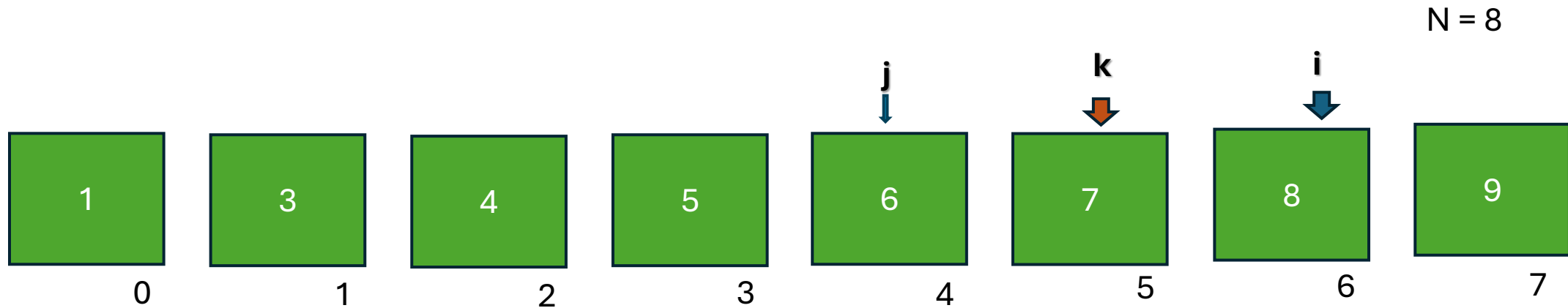
If (7 > 6)
swap

Paso = (paso anterior) / 2

Paso = 1

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 } ;
```



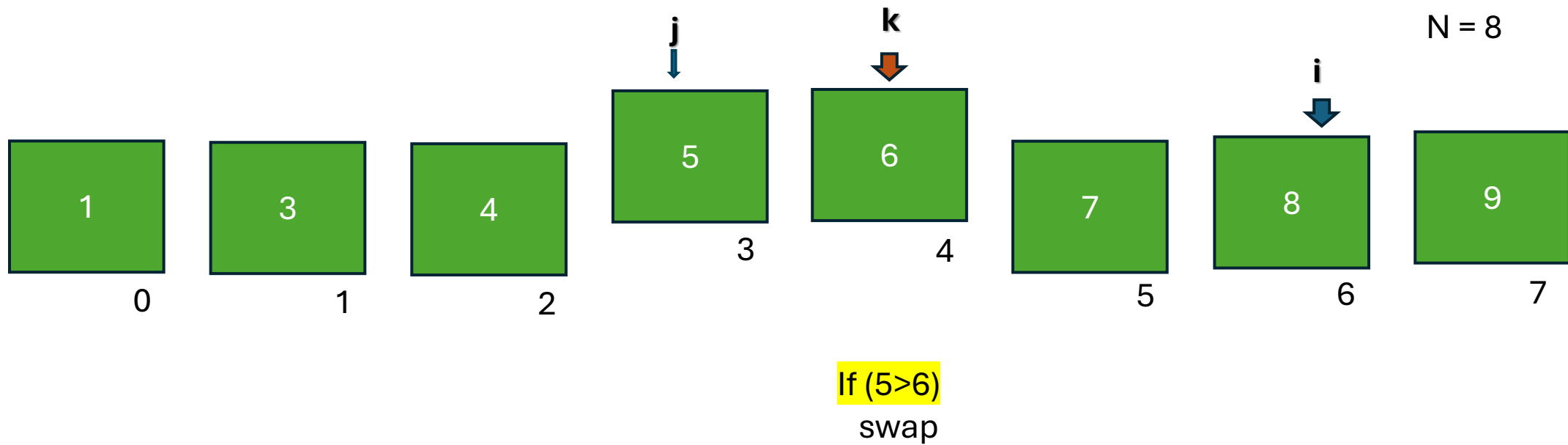
If (7 > 6)
swap

Paso = (paso anterior) / 2

Paso = 1

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```

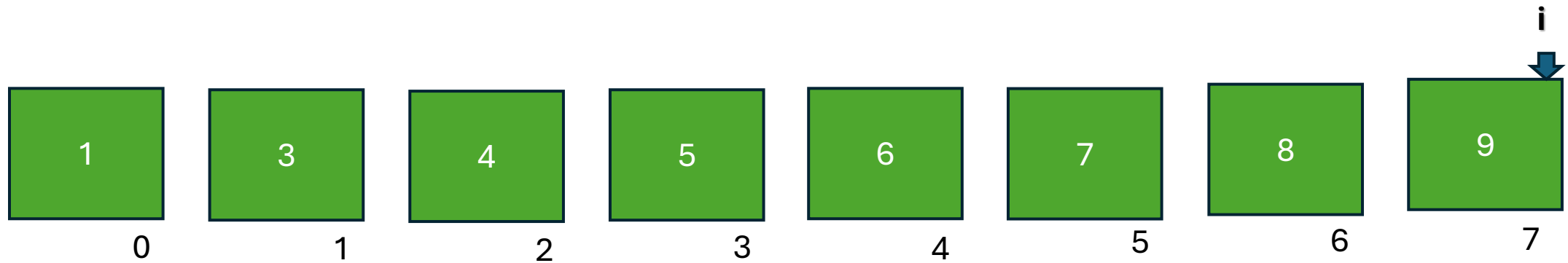


Paso = (paso anterior) / 2

Paso = 1

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```

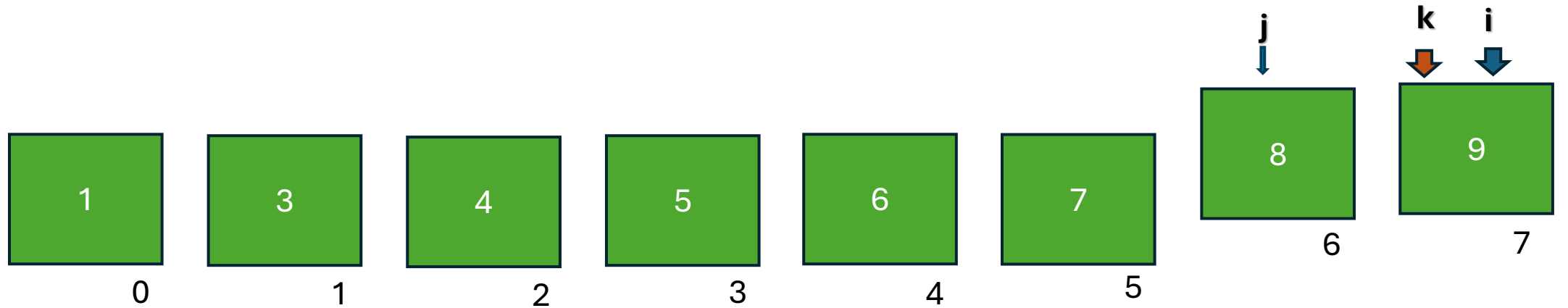


Paso = (paso anterior) / 2

Paso = 1

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```



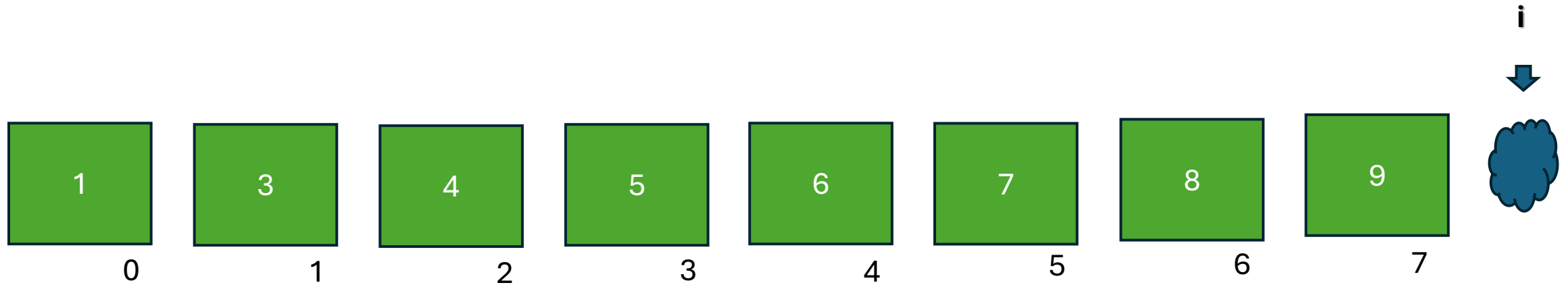
If (8 > 9)
swap

Paso = (paso anterior) / 2

Paso = 1

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 } ;
```

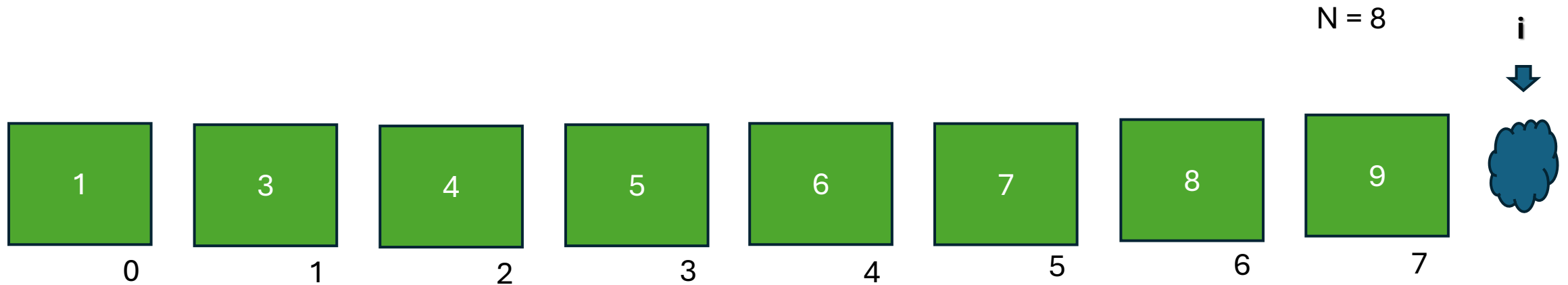


Al terminar de recorrer la estructura se debe recalcular el paso y volver a comenzar

! finaliza al recorrer el ultimo elemento del vector

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 } ;
```



Al terminar de recorrer la estructura se debe recalcular el paso y volver a comenzar

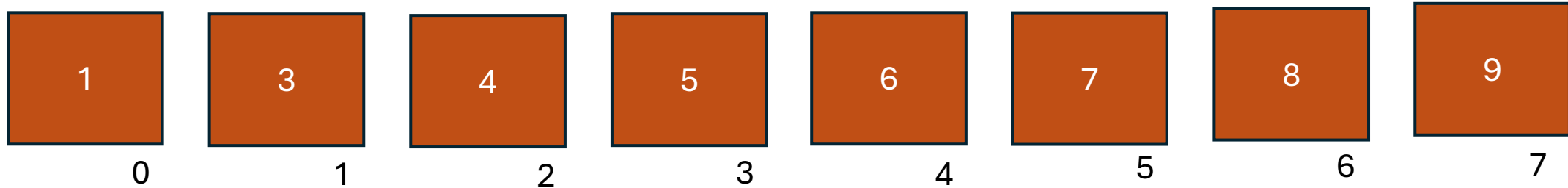
Paso = (paso anterior) / 2
Paso = 0 (solo parte entera)

Cuando el paso llega a cero, termina el ordenamiento

Shell sort

```
int vec[5] = { 3 , 9 , 1 , 8 , 5,7,6,4 };
```

N = 8



Paso = (paso anterior) / 2
Paso = 0 (solo parte entera)

Cuando el paso llega a cero, termina el ordenamiento

Shell sort

```
GNU nano 7.2
/* Shell Sort - version insercion */
#include <stdio.h>

int main()
{
    int i, j, aux, paso;
    int vec[] = { 3, 9, 1, 8, 5, 7, 6, 4 };
    int length = sizeof(vec) / sizeof(vec[0]);

    for (paso = length/2; paso > 0; paso = paso/2)
    {
        for (i = paso; i < length; i++)
        {
            aux = vec[i];
            j = i;
            while (j >= paso && vec[j-paso] > aux)
            {
                vec[j] = vec[j-paso];
                j -= paso;
            }
            vec[j] = aux;
        }
    }

    for (i = 0; i < length; i++)
        printf("%d\n", vec[i]);
    return 0;
}
```

A pesar los tres bucles **anidados (un while, un for y otro while)**, Shell es más eficiente que los algoritmos basicos. Sin embargo, la evaluación de la complejidad de Shell no es directa.

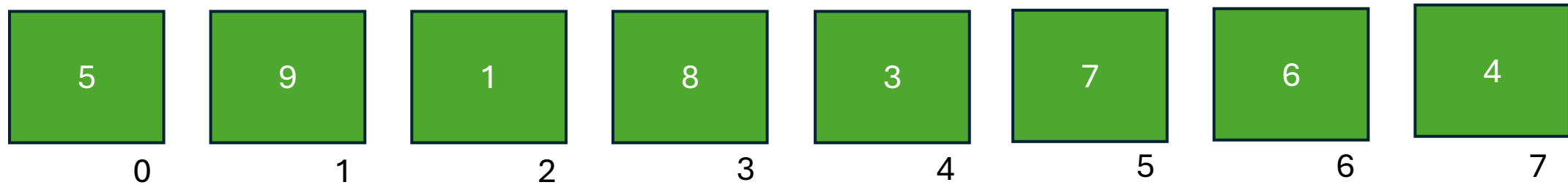
El creador del algoritmo, D. L. Shell, sugiere que el primer salto sea $\frac{n}{2}$ y luego continuar dividiendo este salto por la mitad hasta alcanzar un salto de tamaño 1 (como hicimos en el ejemplo).

Con esta estrategia, se puede demostrar que el tiempo de ejecución es $O(n^2)$ en el peor de los casos, mientras que en promedio es $O(n^{3/2})$.

Este método es superado por otros, por ejemplo quick sort.

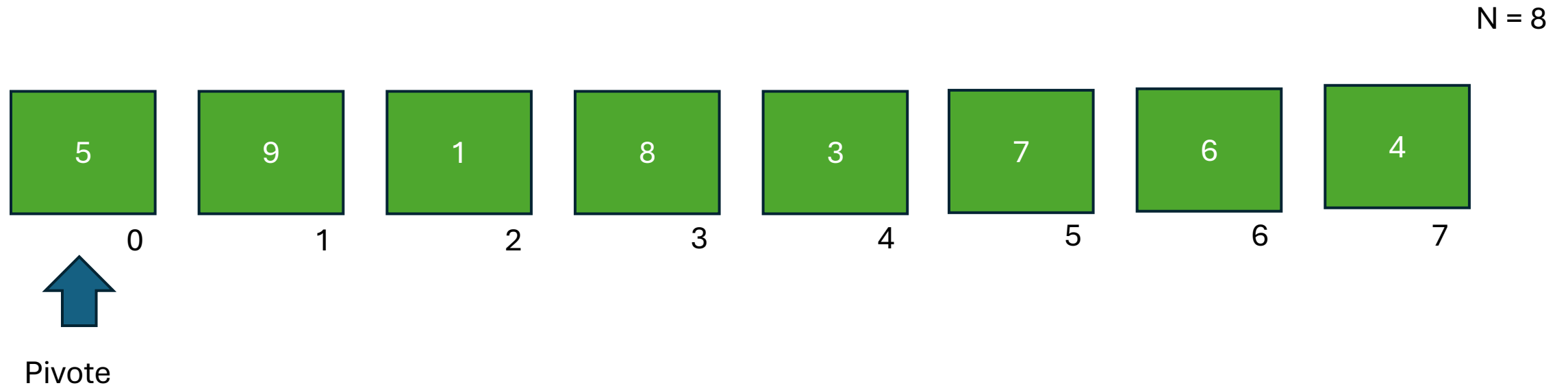
Quick sort -> “Divide y vencerás”

N = 8



DISCLAIMER: Primero veamos la idea principal, después analizamos mas en detalle. Lo que mostramos ahora no va a ser tal cual nuestro código.

Quick sort



Elegimos un elemento cualquiera de la lista, para usar de “pivote”

Quick sort

N = 8



Pivot



Quick sort

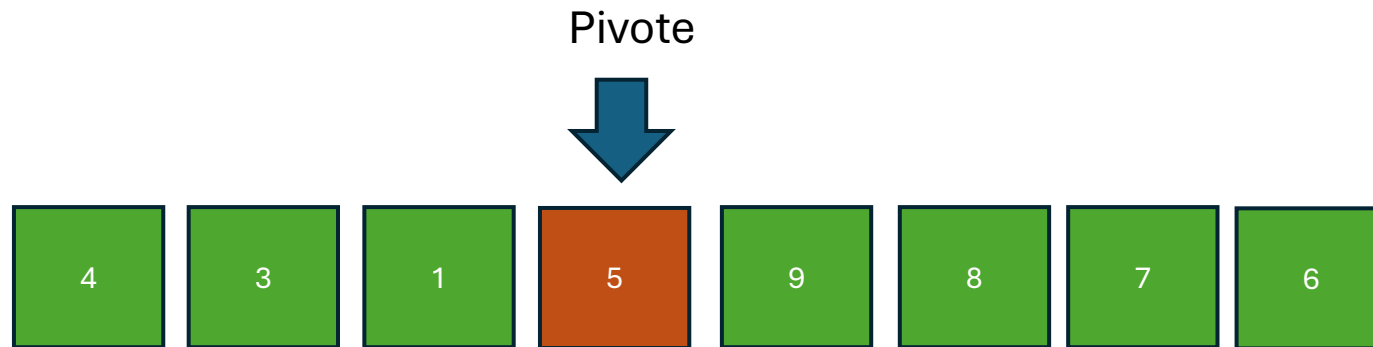
N = 8

¿Qué método usar?
Claro que QuickSort

RECURSIVIDAD

Hasta ahora quick sort recibió una estructura desordenada y solo pudo ubicar un elemento. A este procedimiento lo llamamos:
1 – Ubicar Pivote

Todavía falta:
2- Ordenar estructura izq
3- Ordenar estructura der



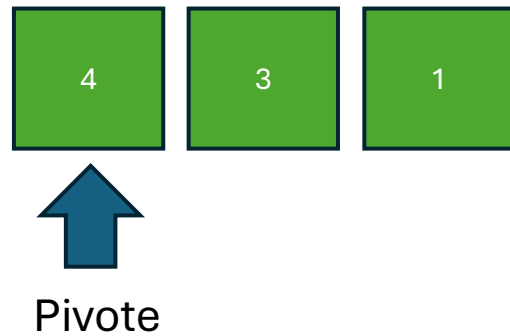
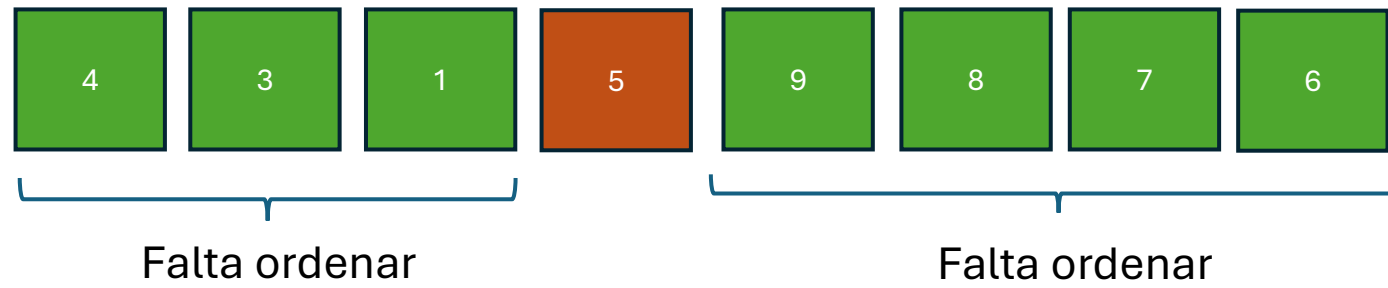
A izq están los
valores inferiores al
pivote

El pivote queda ubicado
en el lugar exacto que
debe estar

A der están los
valores superiores
al pivote

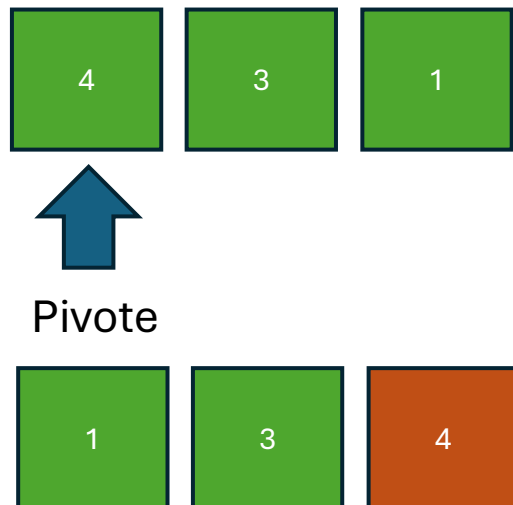
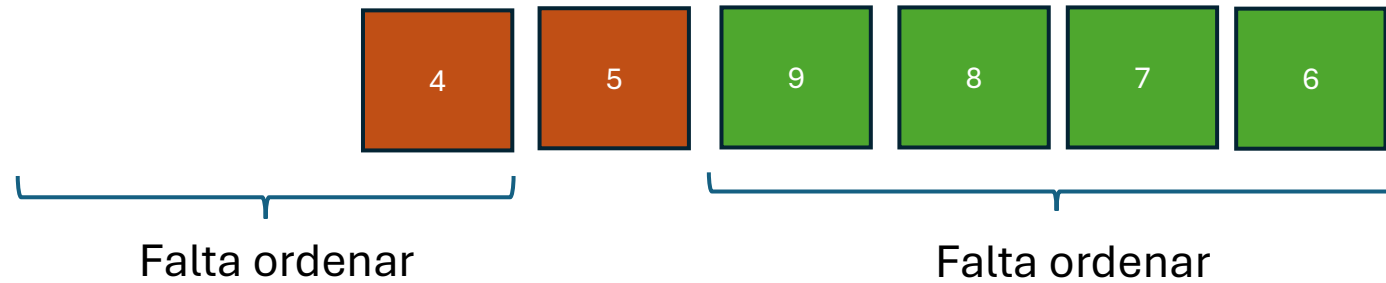
Quick sort

$N = 8$



Quick sort

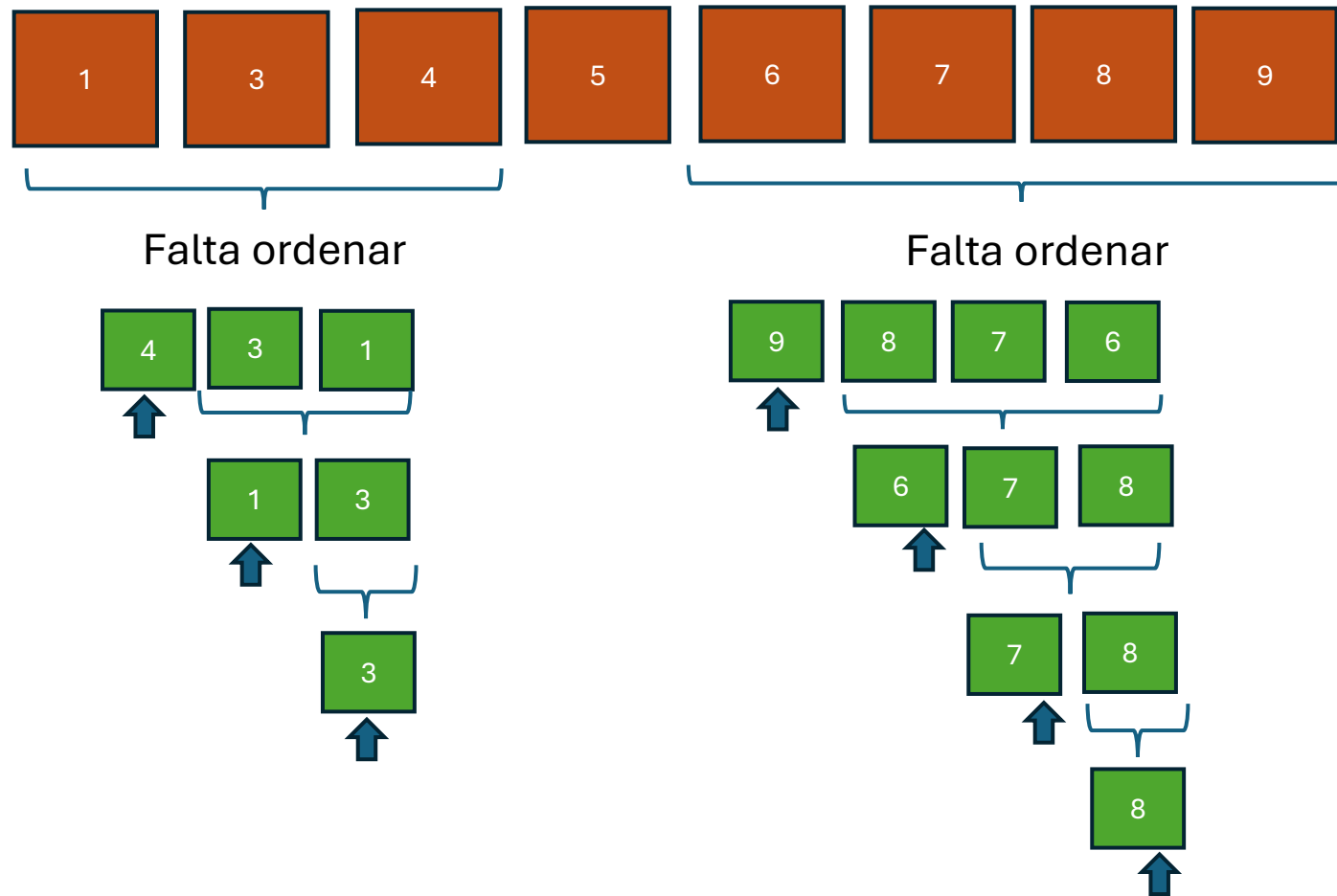
N = 8



Cuando se aplica Quicksort a la parte izquierda de la estructura se busca nuevamente el pivote y este será el segundo elemento correctamente ordenado en la lista.

Quick sort

$N = 8$



Quick sort

Vamos a ver mas detalles de la implementación que vamos a realizar.

En esta etapa de la cursada la estructura mas conocida que manejamos es el vector. Por lo tanto vamos **a ordenar un vector** (mas adelante listas).

Al vector lo vamos a enviar a la función Quicksort por referencia.

```
1  /* Quick Sort*/
2  #include <stdio.h>
3
4  void quicksort(int[], int , int );
5
6  int main()
7  {
8      int i;
9      int vec[] = { 5 , 9 , 1 , 8 , 3 , 7 , 6 , 4 };
10
11      int length = sizeof (vec) / sizeof(vec[0]);
12
13      quicksort(vec , 0, (length-1));
14
15      for (i=0; i< length ; i++)
16      |   printf("%d\n", vec[i]);
17
18      return 0;
19  }
```

La función Quicksort no devuelve nada.

Dentro de los argumentos le enviamos la dirección de memoria del vector que queremos ordenar “vec”.

Por lo tanto todos los cambios que haga la función los va a hacer en el vector original, que es exactamente lo que queremos.

El vector NO se copia dentro de las funciones recursivas.

Quick sort

```
1  /* Quick Sort*/
2  #include <stdio.h>
3
4  void quicksort(int[], int , int );
5
6  int main()
7  {
8      int i;
9      int vec[] = { 5 , 9 , 1 , 8 , 3 , 7 , 6 , 4 };
10
11      int length = sizeof (vec) / sizeof(vec[0]);
12
13      quicksort(vec , 0, (length-1));
14
15      for (i=0; i< length ; i++)
16          printf("%d\n", vec[i]);
17
18      return 0;
19  }
```

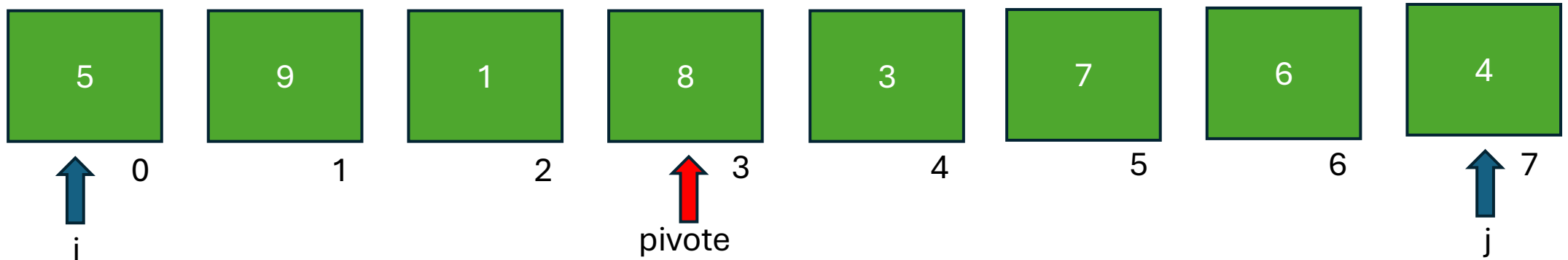
Diagram illustrating the initial call to `quicksort` in `main`. The function signature is `quicksort(int[], int , int );`. The first parameter is the array `vec`. The second parameter, labeled "inicio" (start), is `0`. The third parameter, labeled "fin" (end), is `(length-1)`. Arrows point from the labels "inicio" and "fin" to the second and third arguments of the `quicksort` function call respectively.

La primera vez que usamos la función `quicksort` vamos a enviar el vector completo.

Pero cuando llamemos recursivamente a `quicksort`, vamos a enviar solo una parte de este vector. Por eso es importante definir desde que elemento hasta que elemento vamos a enviar.

Quick sort

Si bien se puede elegir cualquier elemento del vector como pivote, vamos a elegir por razones de eficiencia el elemento central.

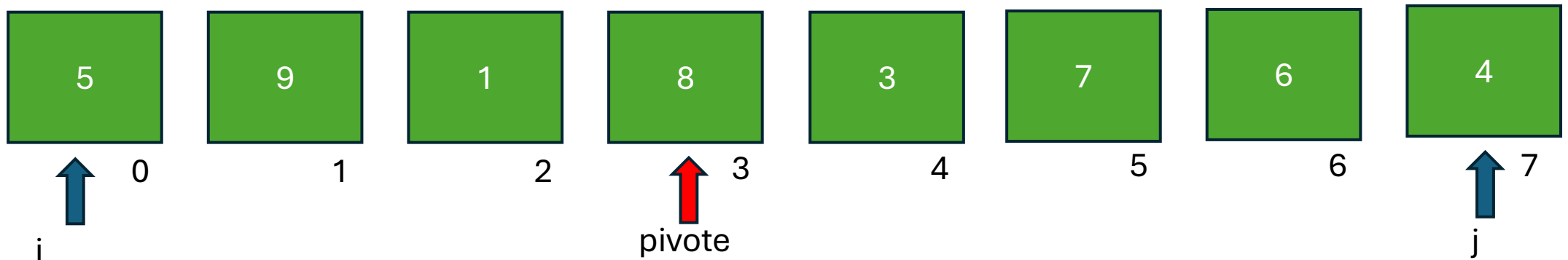


$$posPivote = \frac{fin + inicio}{2} = 3$$

Pivote = 8

Quick sort

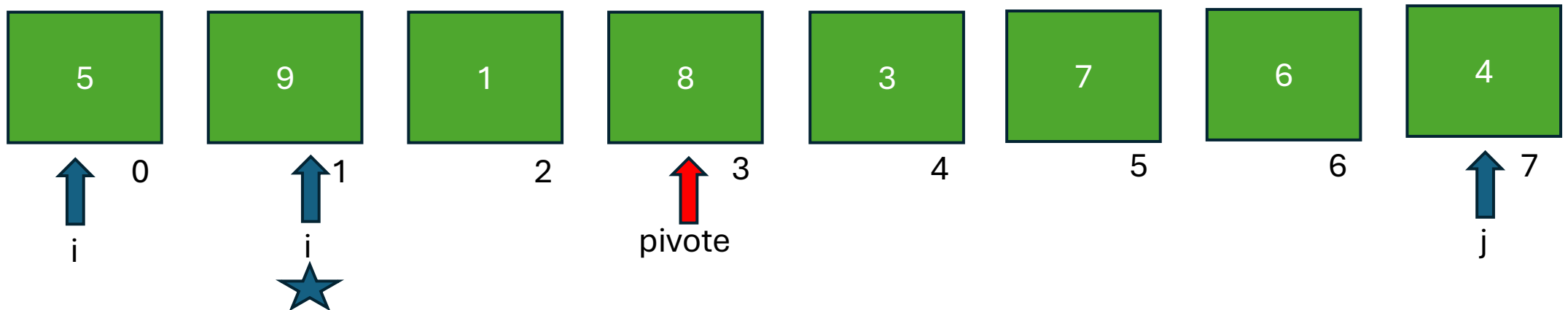
Ahora vamos a recorrer la parte izquierda hasta encontrar el primer valor que no cumpla la condición de ser menor al pivote.



$5 < 8$? Si. Por lo tanto muevo i una posición

Quick sort

Ahora vamos a recorrer la parte izquierda hasta encontrar el primer valor que no cumpla la condición de ser menor al pivote.

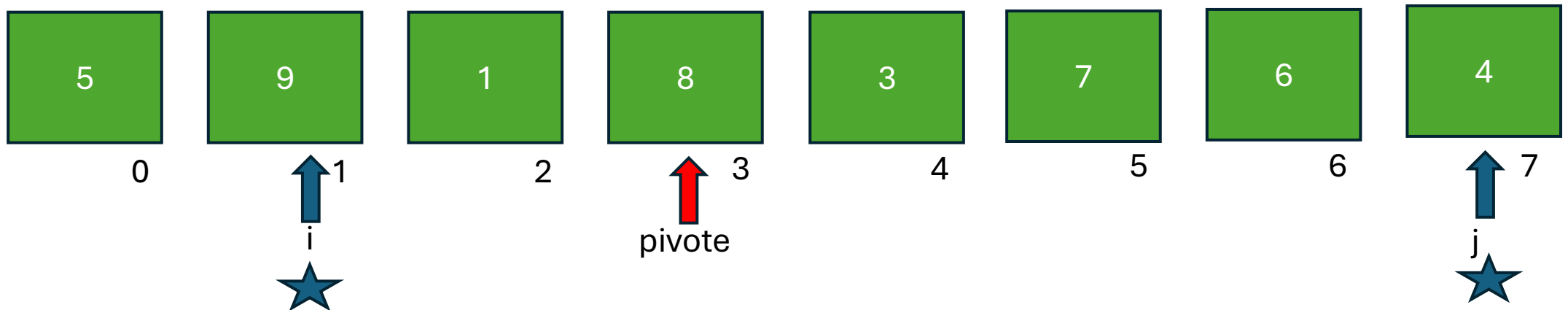


$9 < 8$?

No. Me detengo acá. Donde apunta i hay un valor que debe estar del otro lado.

Quick sort

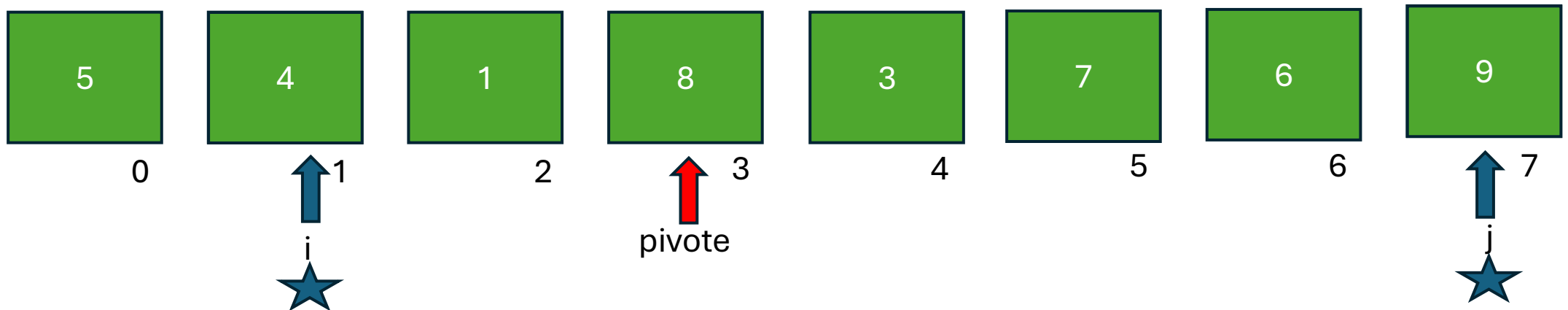
Ahora vamos a recorrer la parte derecha hasta encontrar el primer valor que no cumpla la condición de ser menor al pivote.



$4 > 8$? No. Me detengo acá. Tengo un valor que debe ir del lado contrario

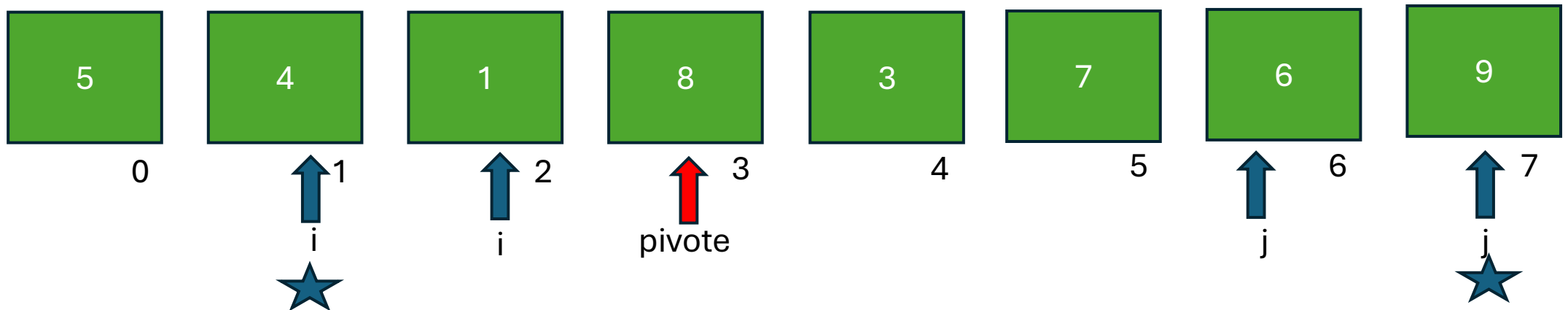
Quick sort

Antes de swapear, verificamos que i y j no se hayan cruzado. Si no se cruzaron hacemos el swap.



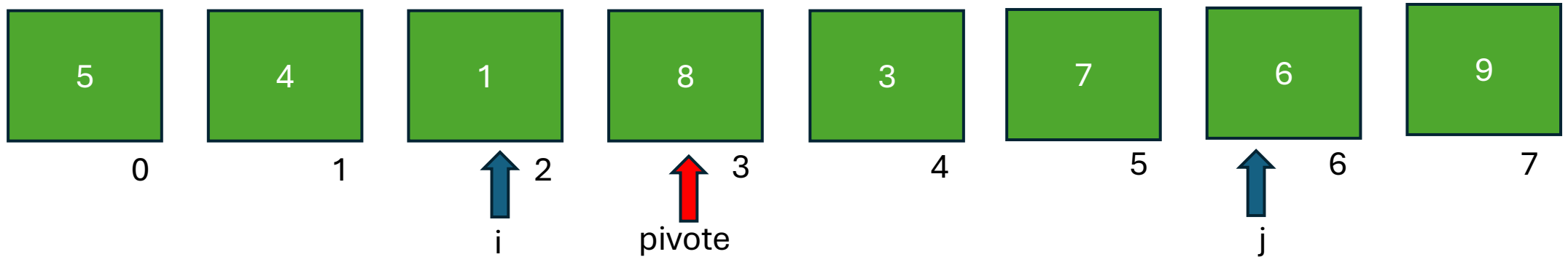
Quick sort

Avanzamos i y retrocedemos j. Para seguir comparando los elementos que faltan.



Verificamos que i y j no se hayan cruzado y seguimos.

Quick sort

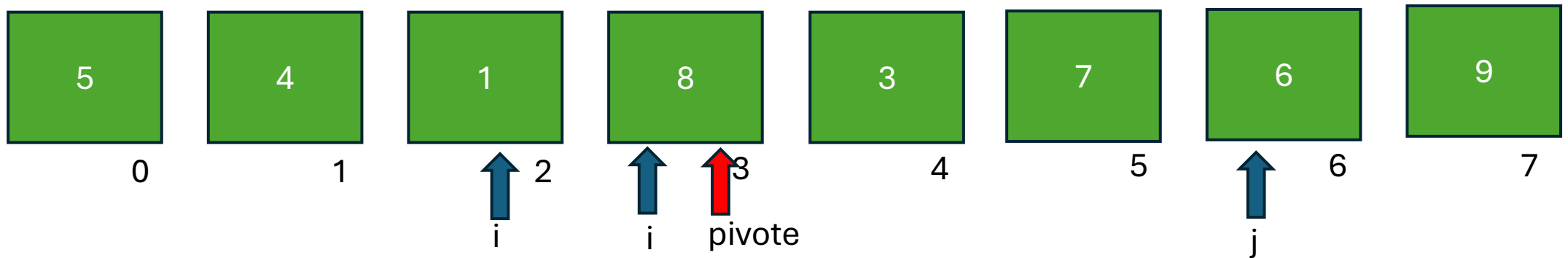


$1 < 8$??

Si, por lo tanto sigo avanzando.

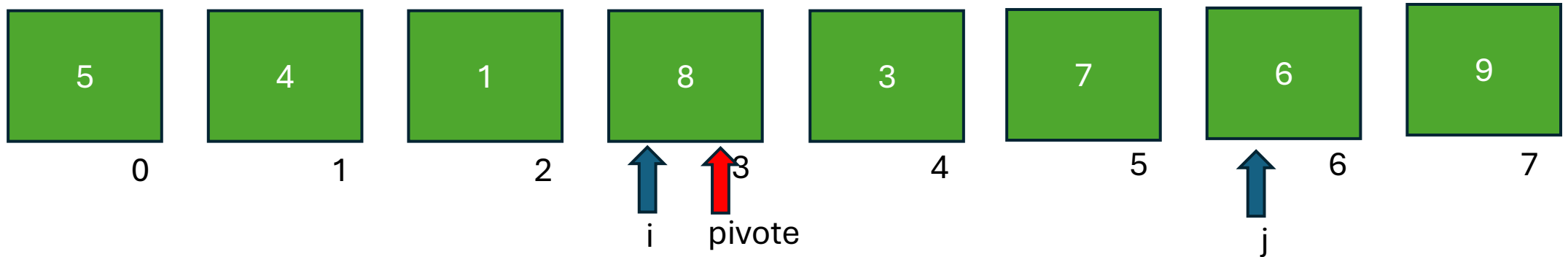
Quick sort

Hay muchas implementaciones para realizar este algoritmo, el caso que estamos mostrando es tal cual el código que vamos a compartir en clase.



$8 < 8$?? No, por lo tanto dejamos i en este valor y seguimos analizando del otro lado.

Quick sort

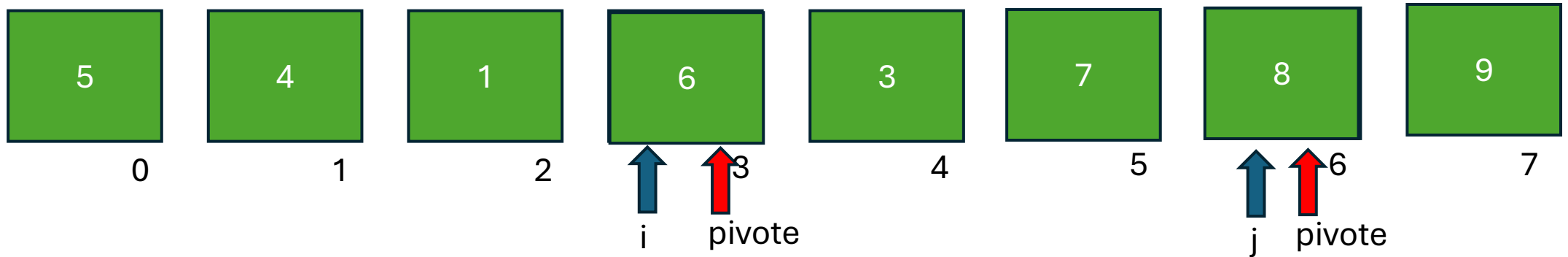


$6 > 8$??

No. Tengo un valor que debe cruzar del otro lado.

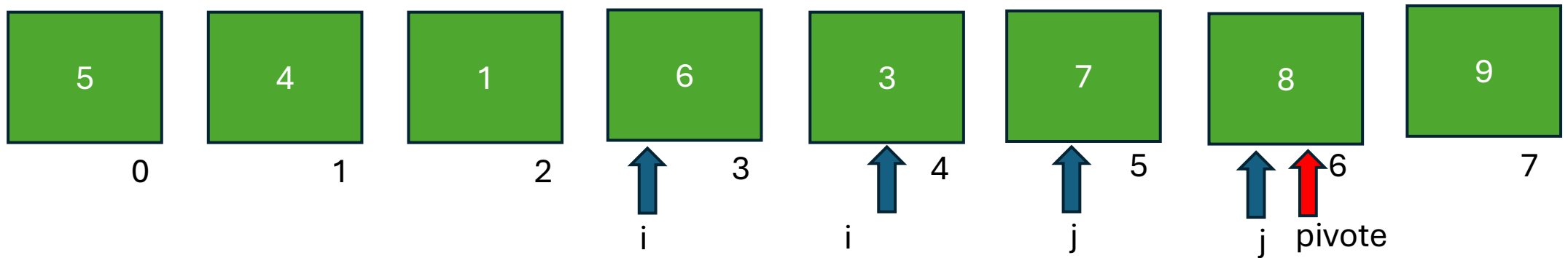
Quick sort

Antes de swapear, verificamos que i y j no se hayan cruzado. Si no se cruzaron hacemos el swap.

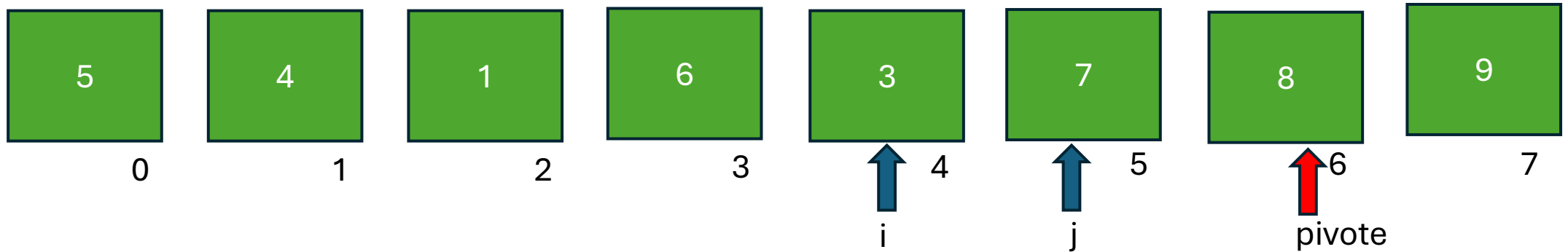


Quick sort

Se actualizan las posiciones de i y j y verificamos que no se crucen antes de continuar.



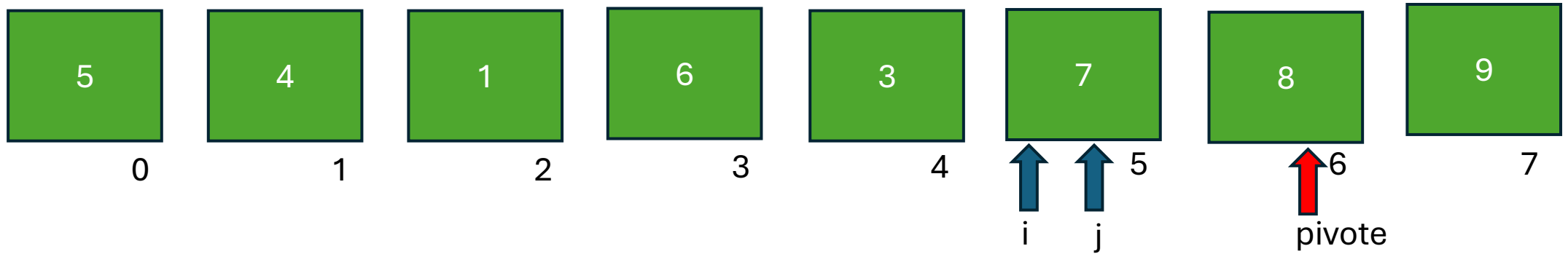
Quick sort



$3 < 8$???

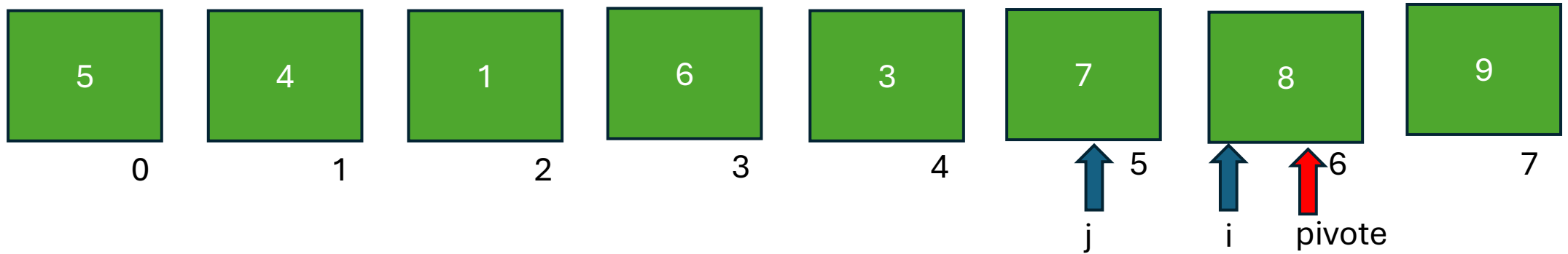
Si, por lo tanto seguimos.

Quick sort



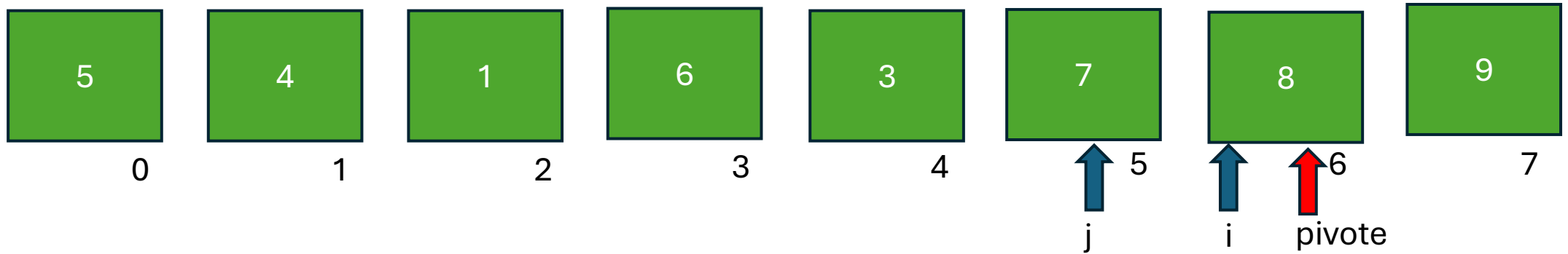
$7 < 8$??? Si. Avanzamos i.

Quick sort



$8 < 8$??? No, nos detenemos. Y vamos a analizar del otro lado.

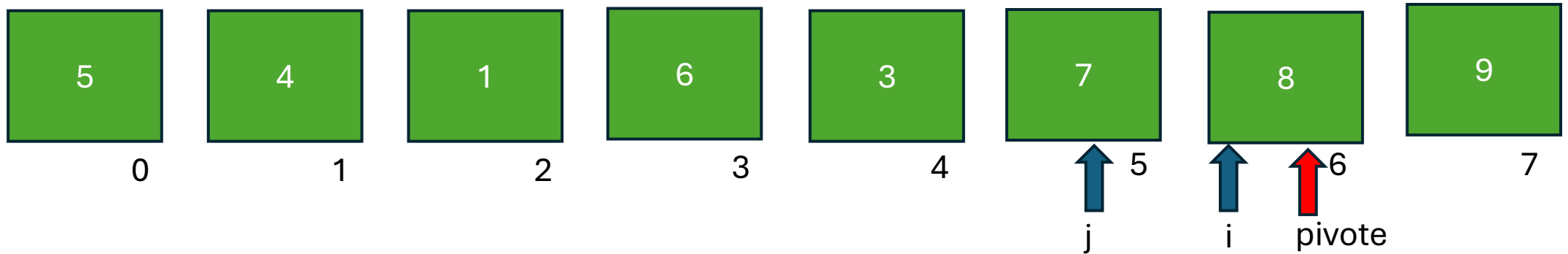
Quick sort



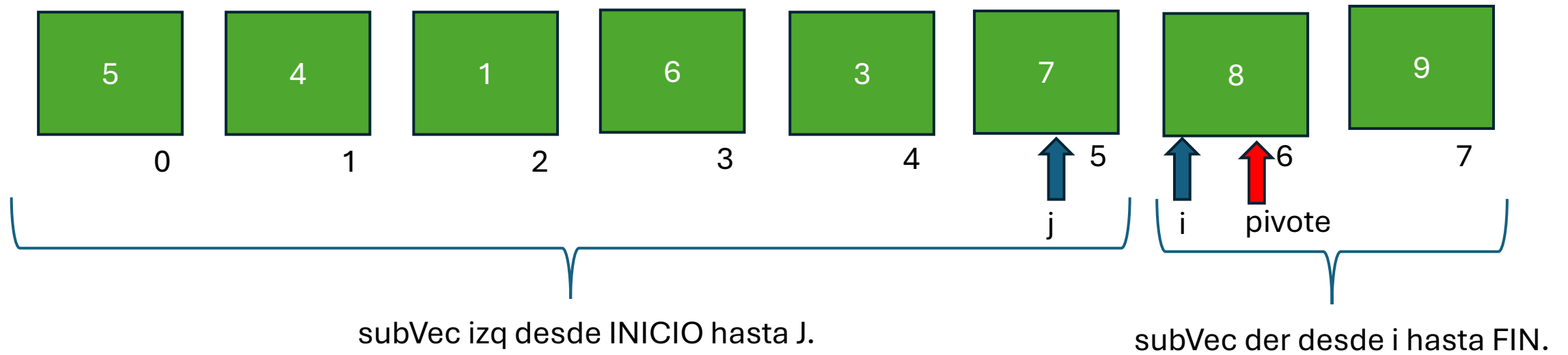
$7 > 8$??? No, nos detenemos.

Quick sort

Cuando verificamos i y j se cruzaron. Por lo tanto se termina este proceso.

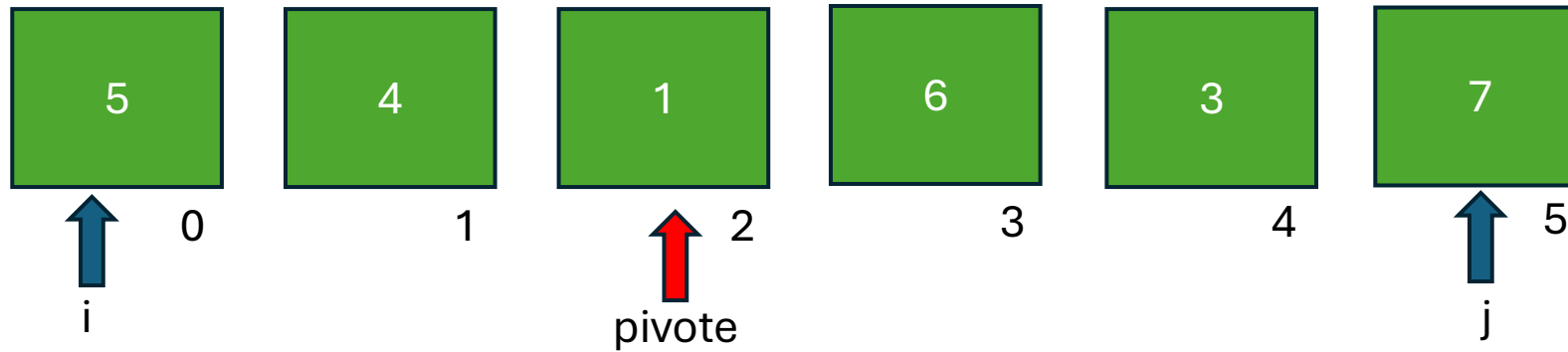


Quick sort



A cada uno de estos vectores se les aplica recursivamente el mismo algoritmo.

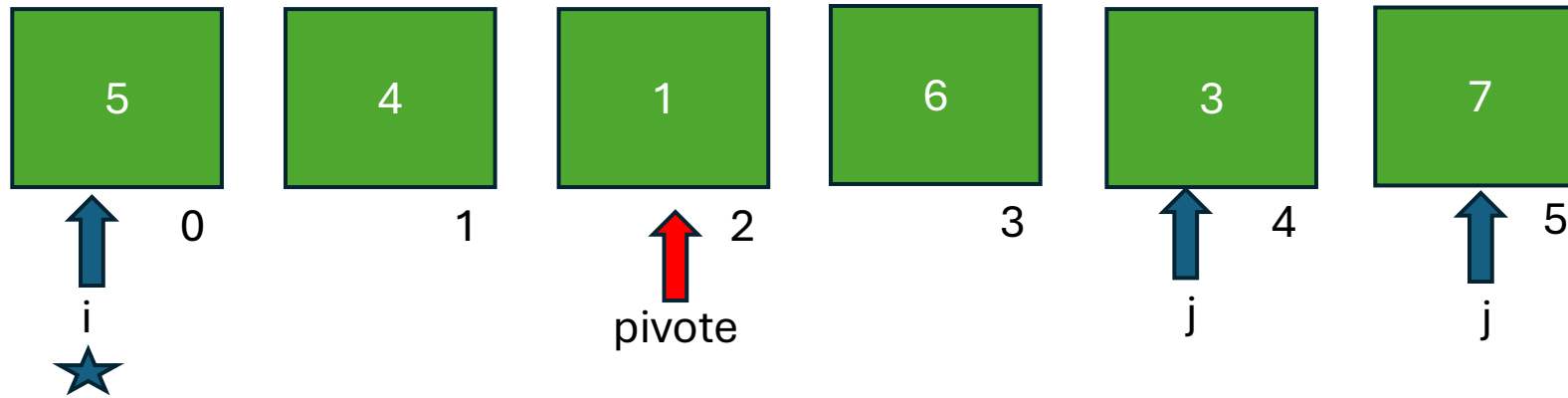
Quick sort – dentro de subVec izq



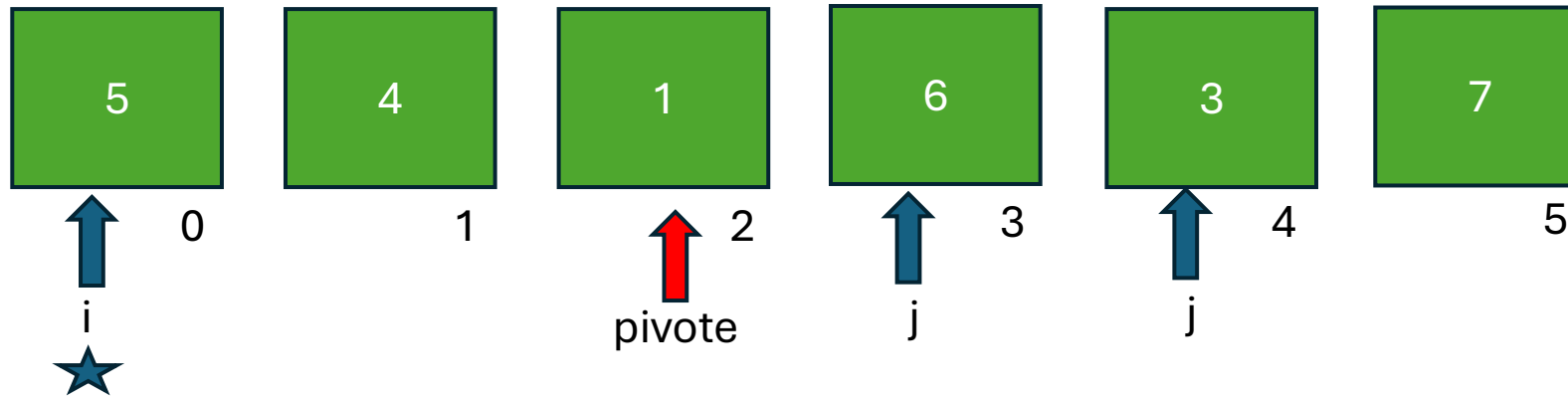
$$posPivot = \frac{fin + inicio}{2} = 2$$

$Pivot = 1$

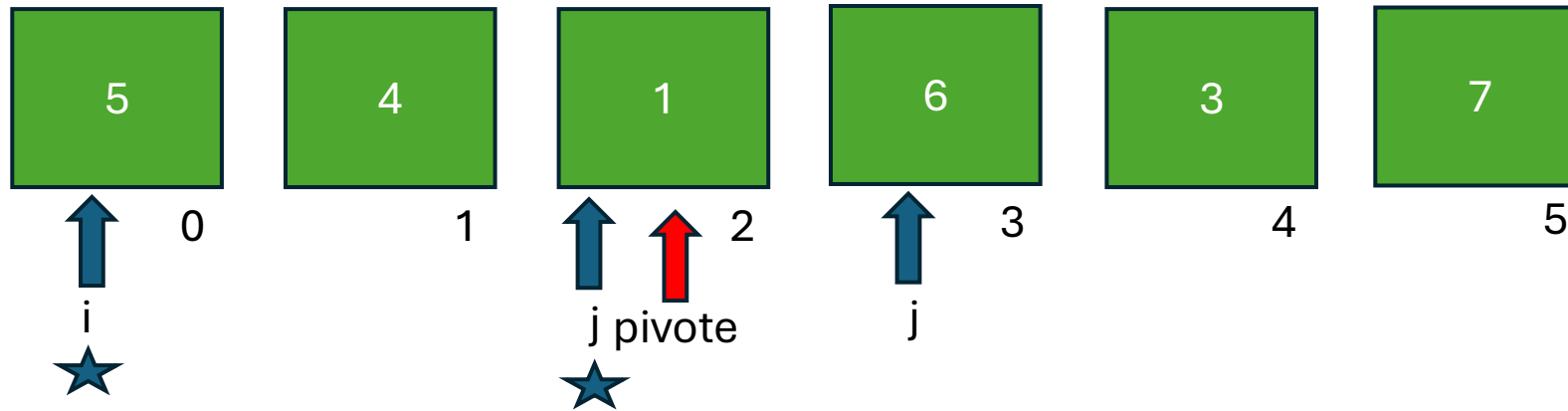
Quick sort – dentro de subVec izq



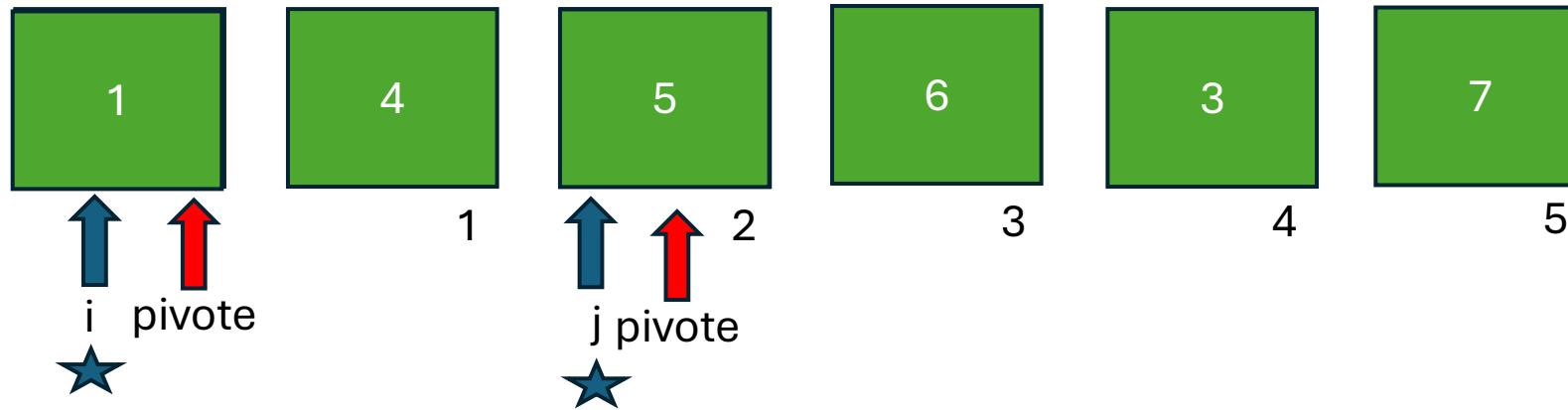
Quick sort – dentro de subVec izq



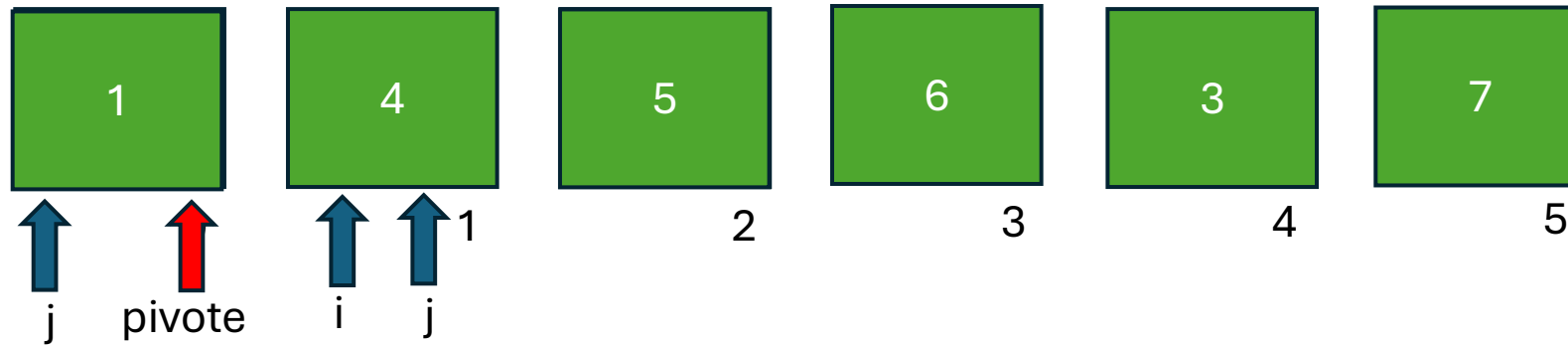
Quick sort – dentro de subVec izq



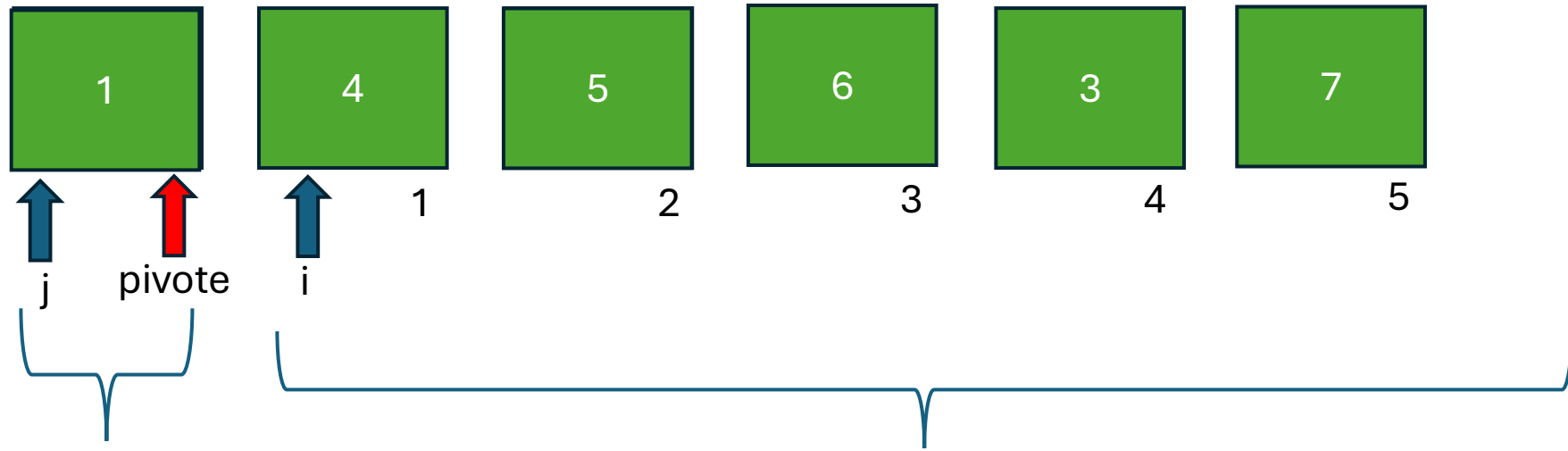
Quick sort – dentro de subVec izq



Quick sort – dentro de subVec izq



Quick sort – dentro de subVec izq



subVec izq desde INICIO hasta J,
como es de un elemento YA esta
ordenado. No vuelve a entrar a la
función recursiva.

subVec der desde i hasta FIN.

Cuando el pivote cae justo en un elemento va a crear una subestructura de un elemento, es el peor caso desde el punto de vista de la eficiencia. Que pase esto consistentemente baja el rendimiento

Quick sort

```
23 void quicksort(int vec[], int inicio, int fin)
24 {
25     int i, j, central;
26     int pivote;
27     int aux;
28     central = (inicio + fin) / 2;
29     pivote = vec[central];
30     i = inicio;
31     j = fin;
32     do {
33         while (vec[i] < pivote) i++;
34         while (vec[j] > pivote) j--;
35         if (i <= j)
36         {
37             aux = vec[i];
38             vec[i] = vec[j];
39             vec[j] = aux;
40
41             i++;
42             j--;
43         }
44     }while (i <= j);
45     if (inicio < j)
46         quicksort(vec, inicio, j); // mismo proceso con sublista izqda
47     if (i < fin)
48         quicksort(vec, i, fin); // mismo proceso con sublista drcha
49
50 }
```


Quick sort

```
23 void quicksort(int vec[], int inicio, int fin)
24 {
25     int i, j, central;
26     int pivote;
27     int aux;
28     central = (inicio + fin) / 2;
29     pivote = vec[central];
30     i = inicio;
31     j = fin;
32     do {
33         while (vec[i] < pivote) i++;
34         while (vec[j] > pivote) j--;
35         if (i <= j)
36         {
37             aux = vec[i];
38             vec[i] = vec[j];
39             vec[j] = aux;
40
41             i++;
42             j--;
43         }
44     }while (i <= j);
45     if (inicio < j)
46         quicksort(vec, inicio, j); // mismo proceso con sublista izqda
47     if (i < fin)
48         quicksort(vec, i, fin); // mismo proceso con sublista drcha
49
50 }
```

La complejidad en el peor caso de Quicksort es $O(n^2)$. Sin embargo, en general, Quicksort tiene una complejidad media de $O(n \log n)$, lo que lo convierte en el algoritmo de ordenación más rápido que vimos en la clase de hoy.

Uso de qsort de la librería standard

Prototipo de la función **qsort**:

```
void qsort(void *, size_t, size_t, int (*)(const void*, const void*));
```

Puntero a void: primer elemento del vector.

1er size_t: cantidad de elementos en el vector [length].

El 2do size_t: tamaño de cada elemento (por ejemplo, sizeof(int)).

Puntero a función: define cómo comparar dos elementos.

Hacemos un ejemplo en el IDE

Conclusiones

Todos los métodos son eficaces. La eficiencia empieza a pesar a medida que aumenta el valor de **n**.

Se suele recomendar que para listas pequeñas, usar los métodos más eficientes: inserción y selección antes que usar un método de intercambio.

Y para listas grandes se recomienda evitar los métodos básicos y utilizar métodos avanzados Shell, Quick, Heap, Merge, etc

+-----+-----+	
Método	Complejidad
+-----+-----+	
Burbuja	n^2
Inserción	n^2
Selección	n^2
Shell	$n^{(3/2)}$
Quicksort	$n \log n$
+-----+-----+	